

Restful API + Ajax 應用

王孝弘

wang@ispan.com.tw

資展國際股份有限公司

上課用專案

https://github.com/HsiaoHungWang/ajax_flask

大綱

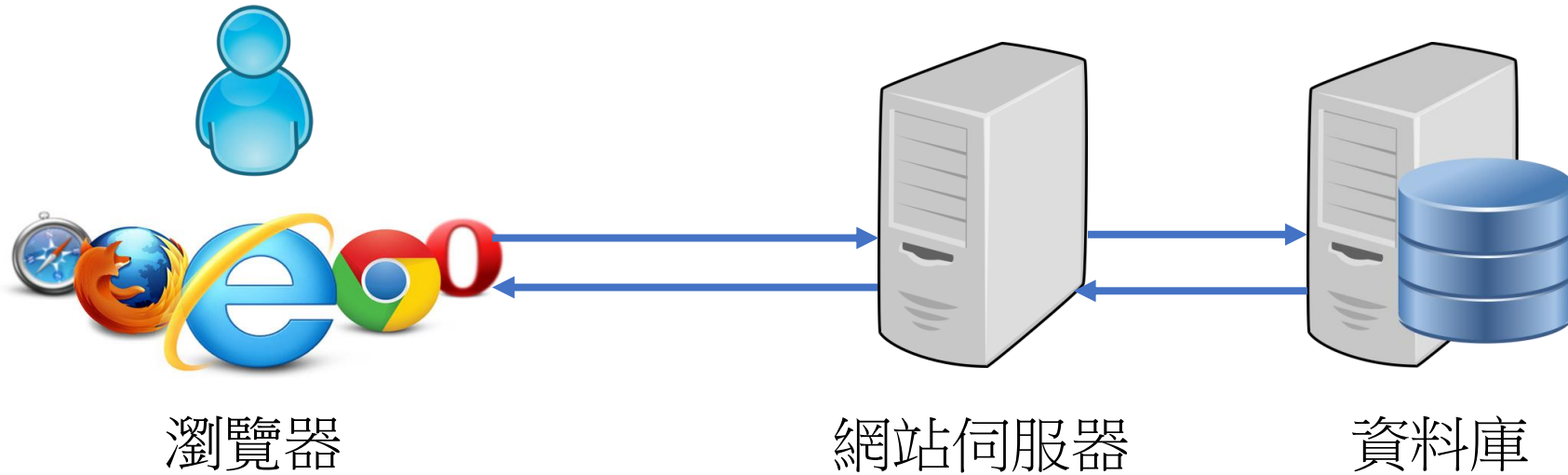


Ajax 開始之前

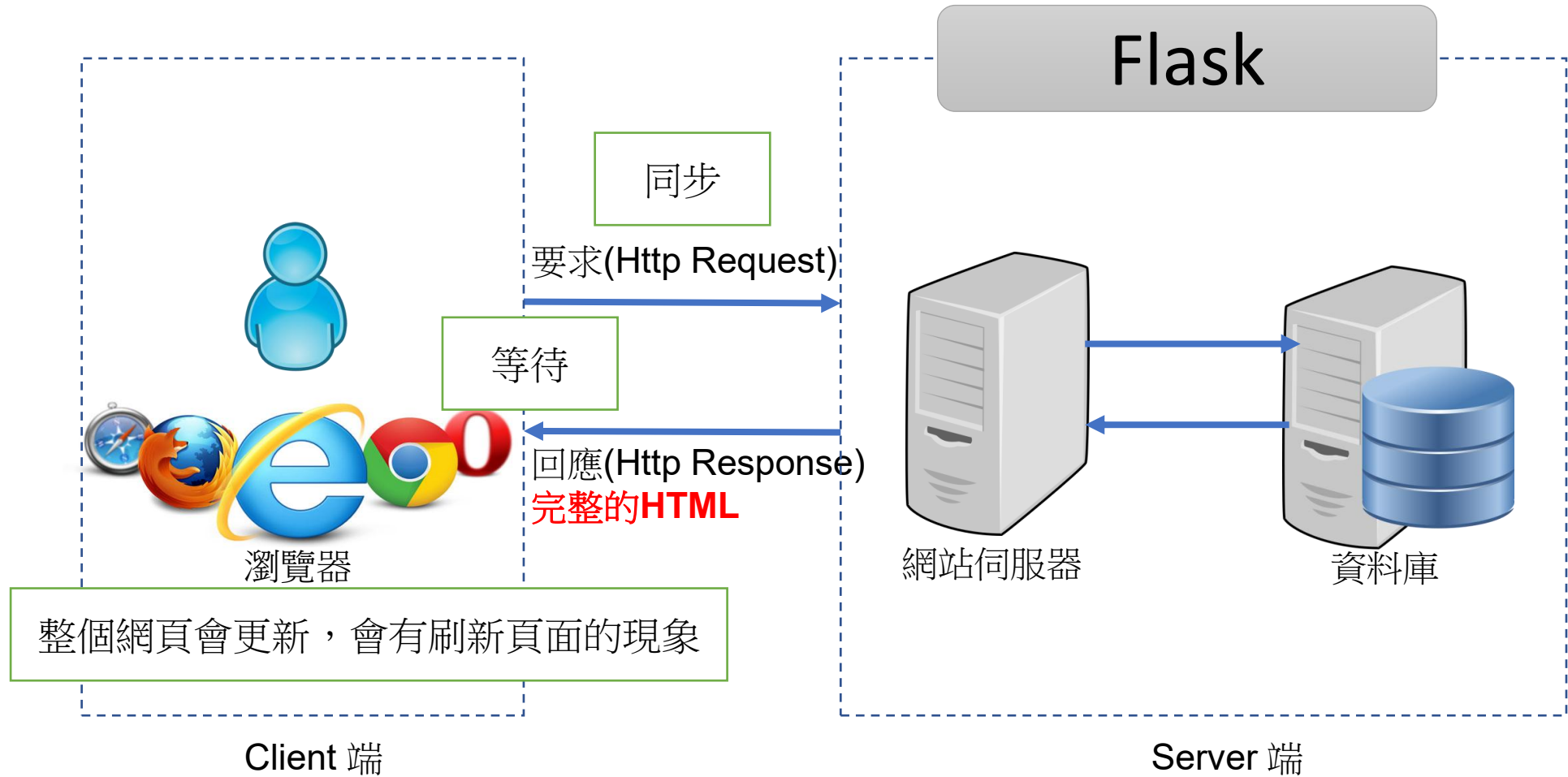
觀念複習

淺談網頁開發

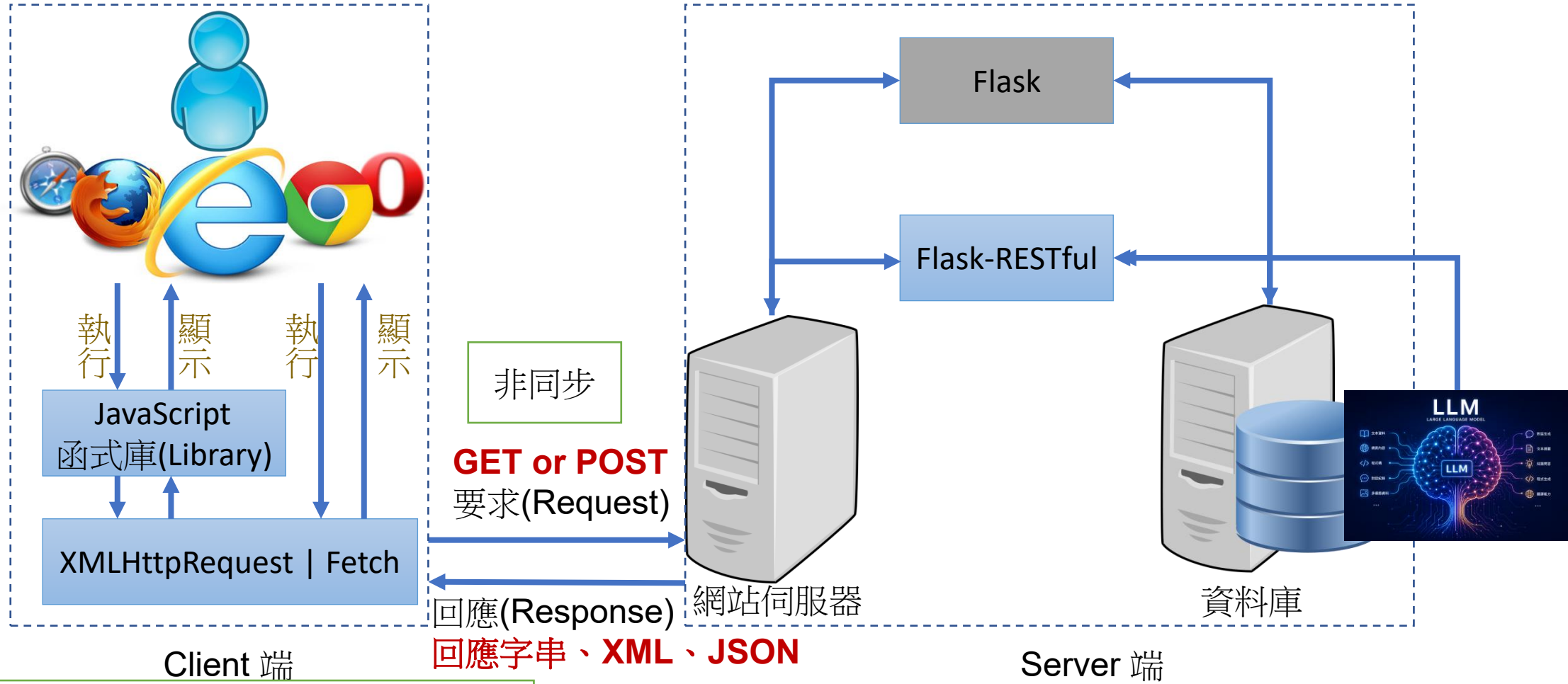
- 傳統的網頁開發
- Ajax的網頁開發
- 即時性的網頁開發



傳統的網頁開發



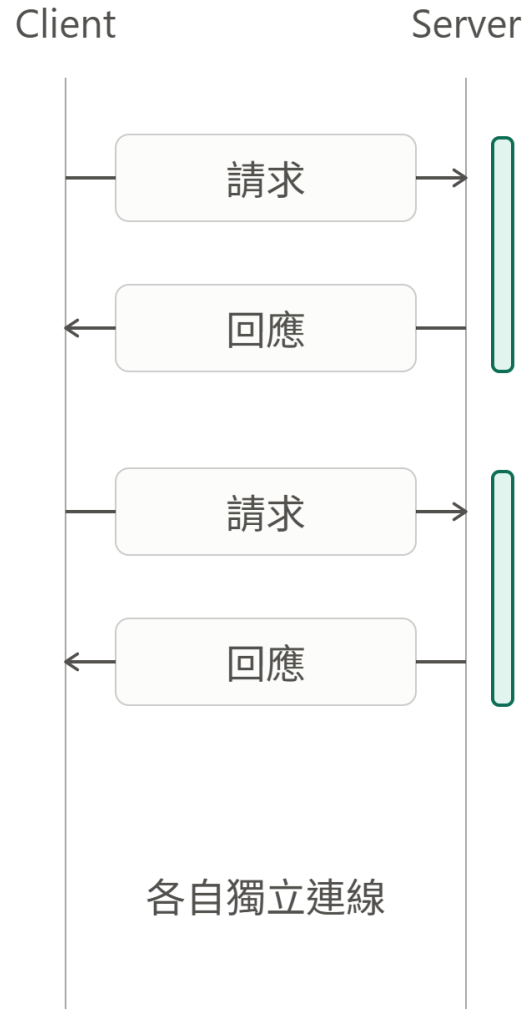
Ajax 網頁開發



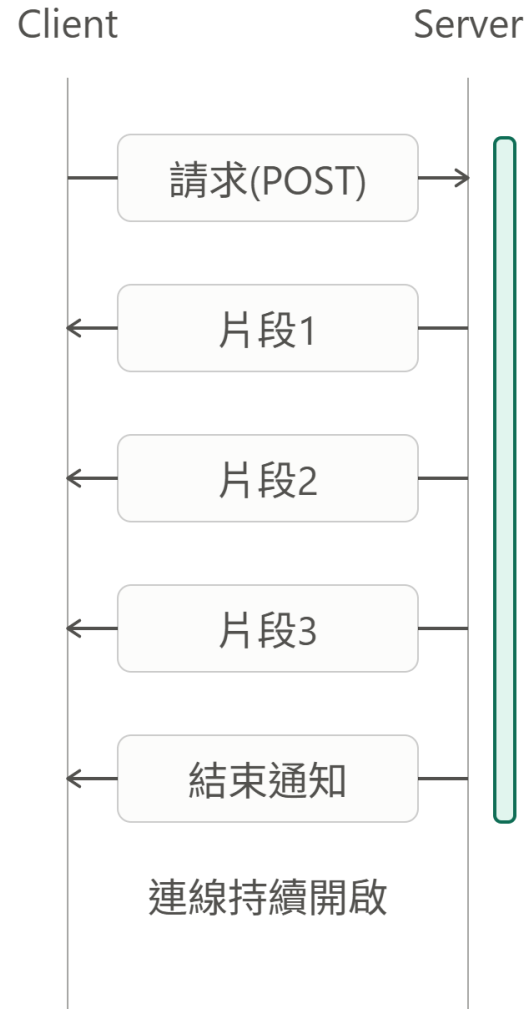
接收資料後，更新網頁上的**部分內容**

即時性網頁開發

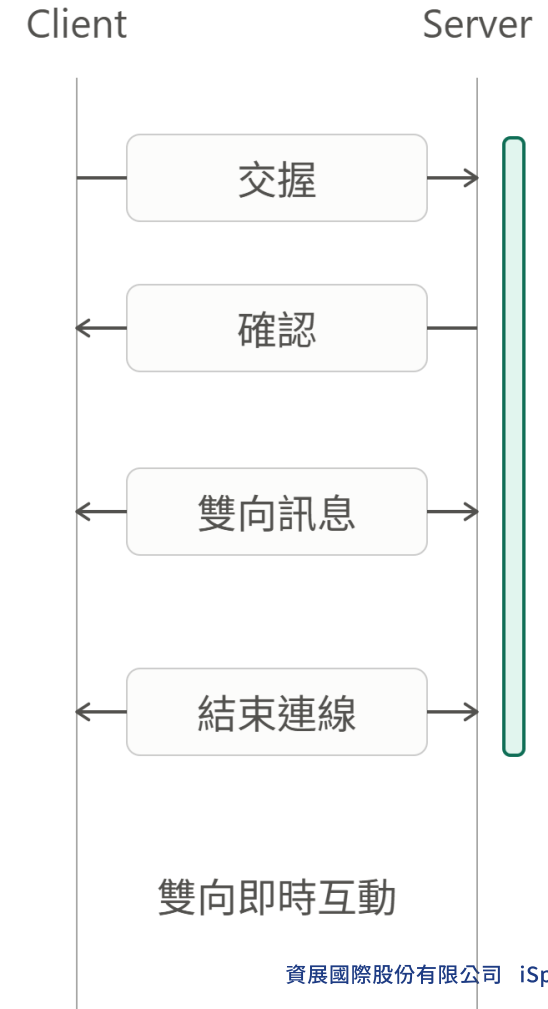
HTTP 一問一答 (AJAX)



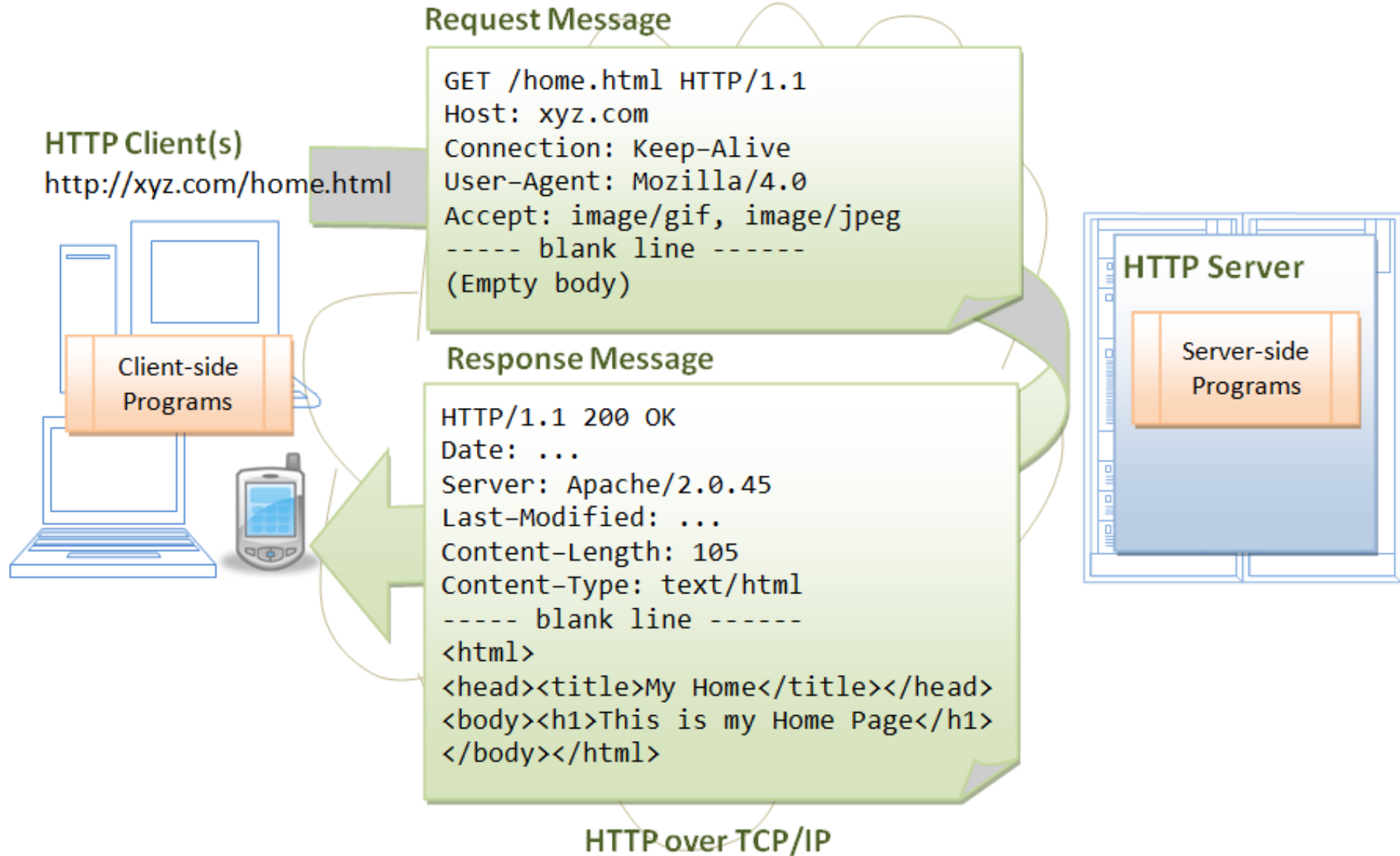
SSE 單向串流



WebSocket 雙向連線



HTTP 通訊協定



AJAX 的由來

- Ajax的概念在 2005/2/18 Jesse James Garrett 所提出
 - <https://web.archive.org/web/20061107032631/http://www.adaptivepath.com/publications/essays/archives/000385.php>
 - 這篇文章中以 Google Suggest 以及 Google Map 為例告訴你，這樣的運用就是 AJAX
 - **A**synchronous(非同步) **J**avaScript **A**nd **X**ML



圖片來自：<https://www.azquotes.com/quote/794357>

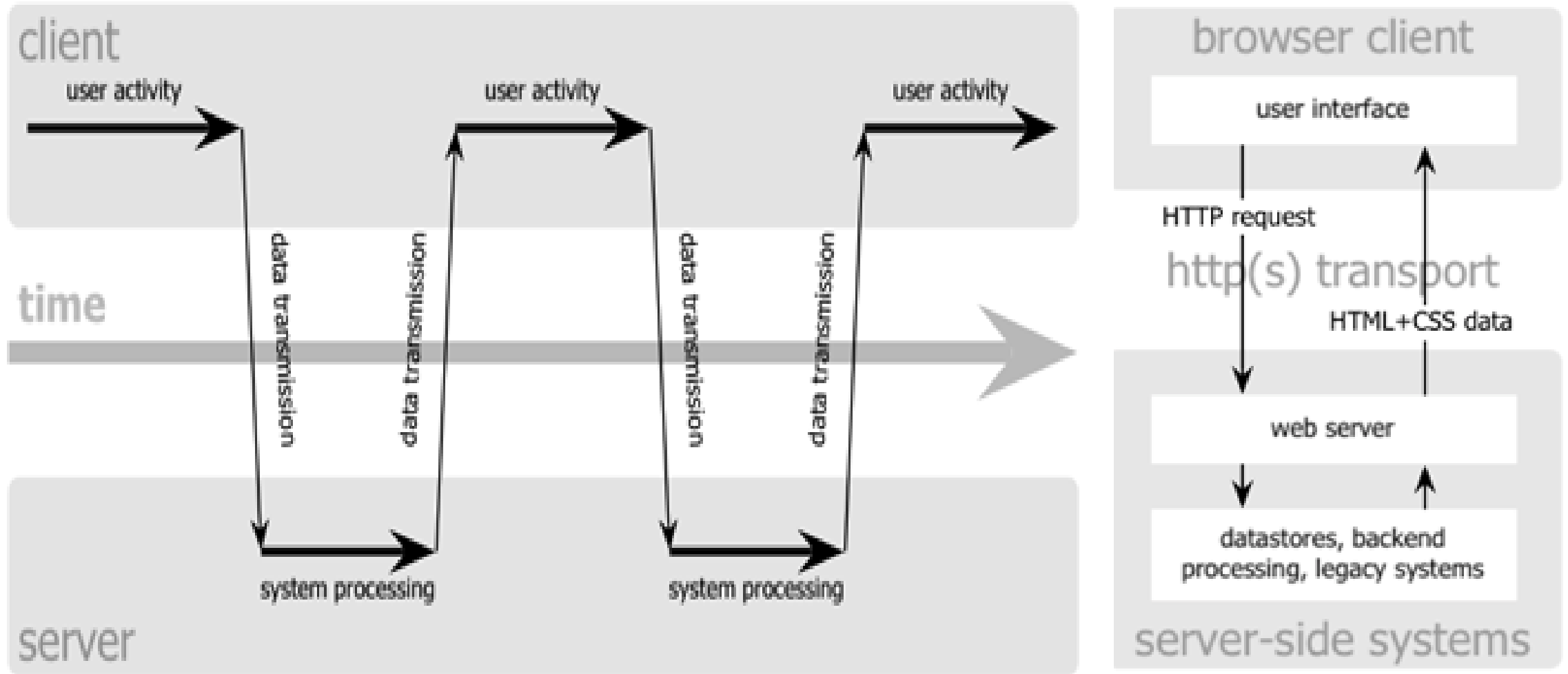
AJAX 的定義

- 不是一個技術
- 是多個技術組合而成的運用
 - 使用標準的 XHTML，CSS 來顯示資料
 - 使用 Document Object Model 的技術與使用者互動
 - 使用 XML及XSLT 交換及傳遞資料
 - 使用 XMLHttpRequest非同步的取得資料
 - 使用 JavaScript 整合所有技術

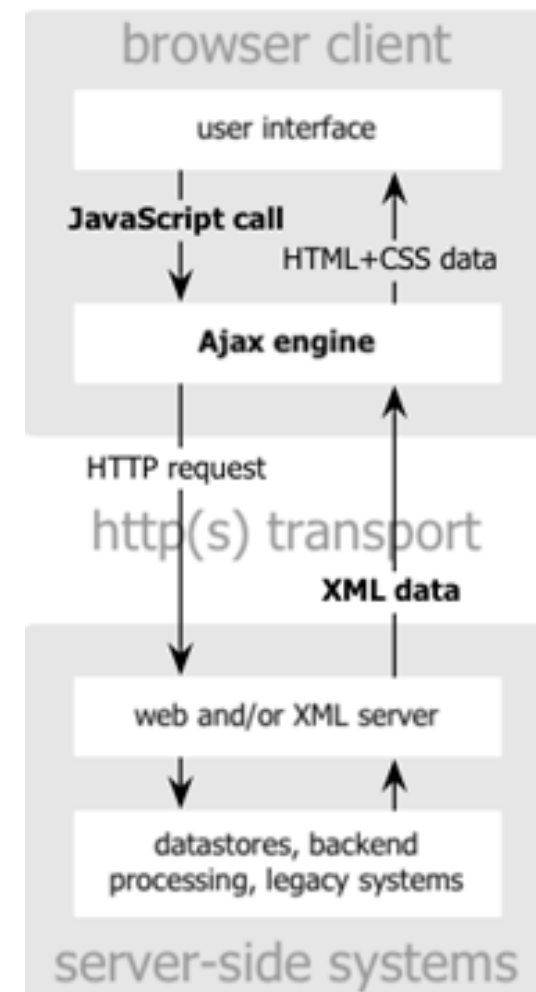
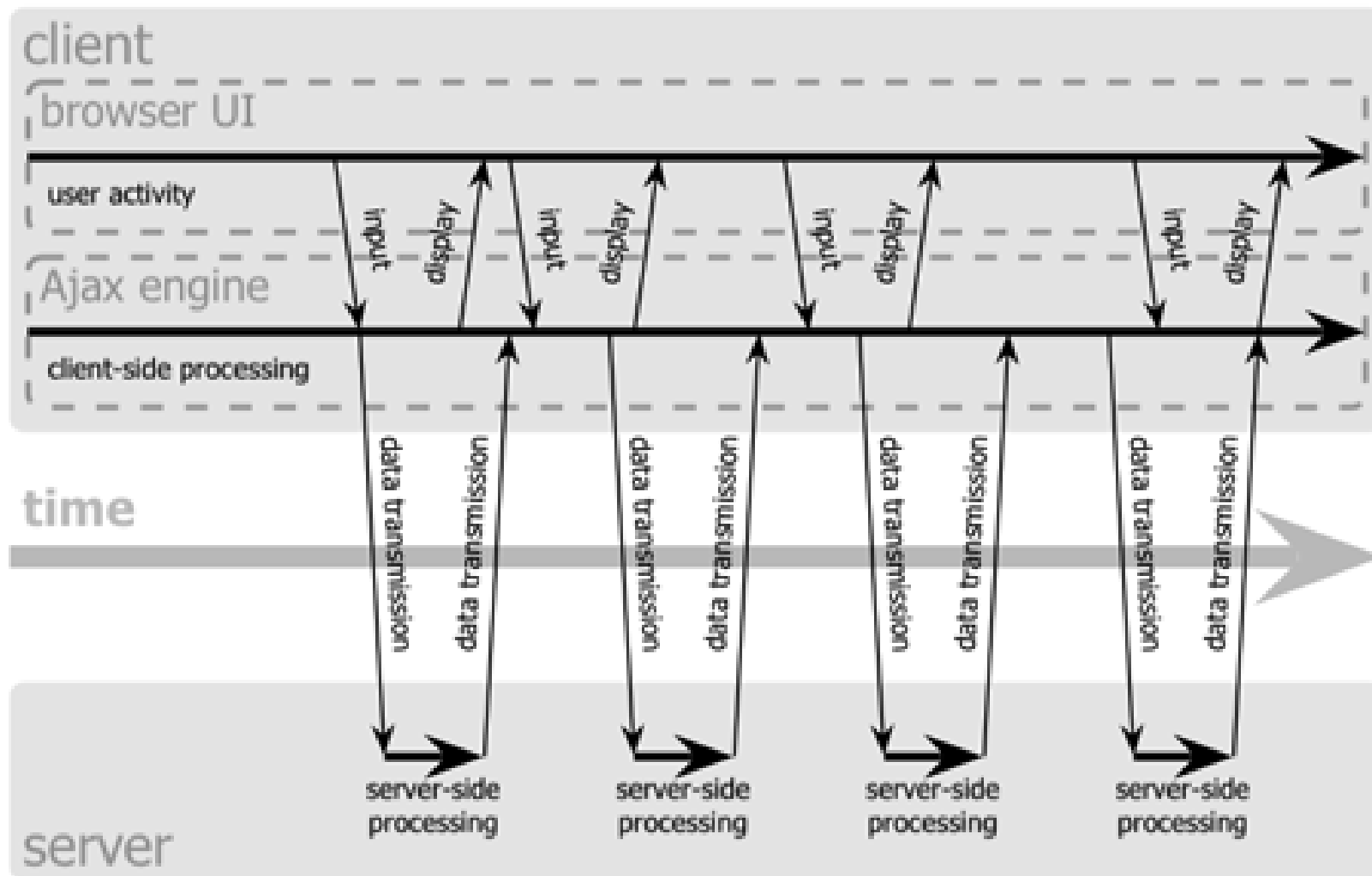
AJAX 的優點

- 不需要切換頁面，可以快速改變頁面
- 可使用不需等待伺服器的非同步請求
- 可將伺服器端的程式搬到客戶端進行
- 減少資料傳輸數量
- 高即時互動性

同步的網站運作模式



非同步的網站運作模式



AJAX 的建立方式

- 完全手工自己打造AJAX
 - JavaScript + DOM + XMLHttpRequest | fetch()
 - 但可能對大多數人而言太困難也太耗時
- 使用AJAX現成函式庫
 - 熟悉函式庫，所提供的功能
 - 建置簡單，快速容易
 - 熱門函式庫
 - jQuery
 - Axios

想想看

網頁開發時，什麼時候會用 Flask的技術?什
麼時候會用 Ajax 技術

你喜歡哪一段程式碼？

(A)

```
蒸蛋(() => {  
  煮咖啡(() => {  
    烤土司(() => {  
      吃早餐()  
    })  
  })  
})
```

(B)

```
蒸蛋()  
  .then(() => 煮咖啡())  
  .then(() => 烤土司())  
  .then(() => 吃早餐())  
  .catch(handleErr)
```

(C)

```
await 蒸蛋()  
await 煮咖啡()  
await 烤土司()  
吃早餐()
```

同步 vs 非同步

刷牙3分鐘

蒸蛋10分鐘

烤土司4分鐘

洗臉3分鐘

煮咖啡5分鐘

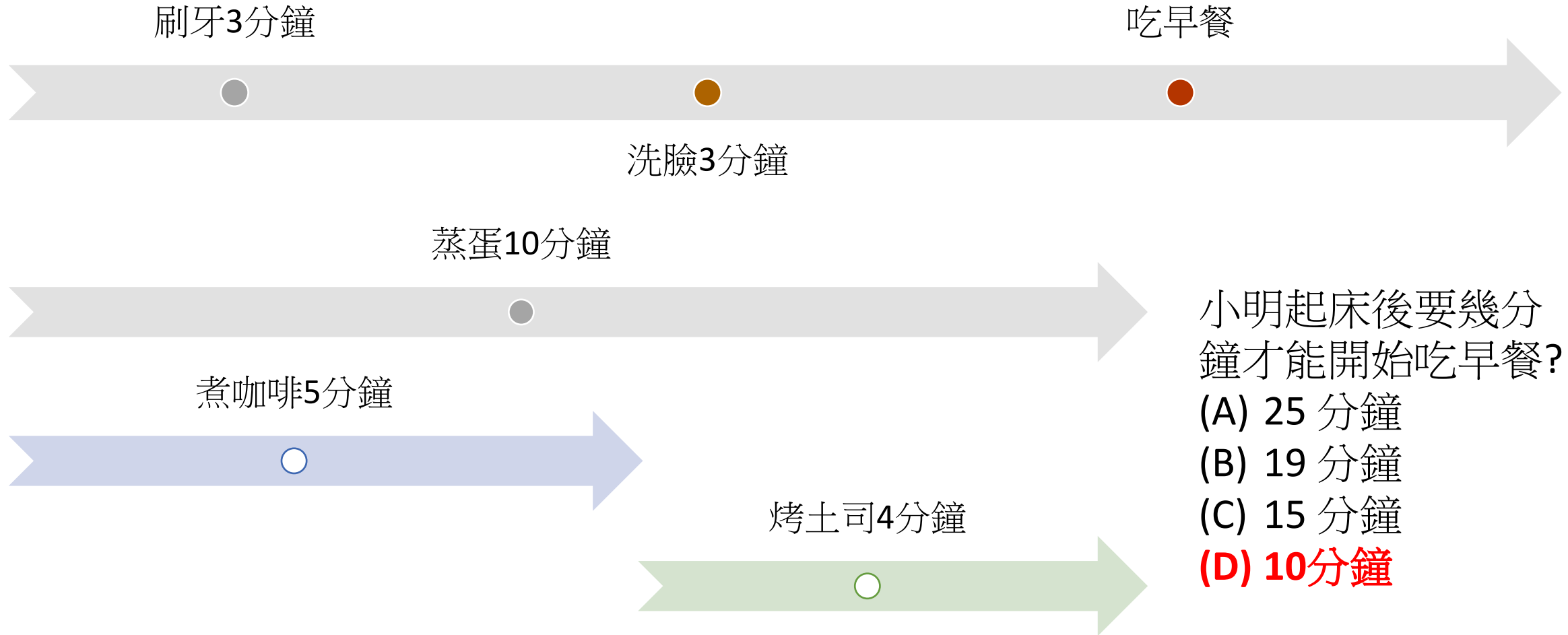
吃早餐

```
刷牙()           //3分鐘  
洗臉()           //3分鐘  
const 蒸蛋完成 = 蒸蛋() //10分鐘  
const 煮咖啡完成 = 煮咖啡() //5分鐘  
const 烤土司完成 = 烤土司() //4分鐘  
If(蒸蛋完成 && 煮咖啡完成 && 烤土司完成 ){  
    吃早餐()  
}
```

小明起床後要幾分鐘才能開始吃早餐?

- (A) 25 分鐘
- (B) 19 分鐘
- (C) 15 分鐘
- (D) 10分鐘

同步 vs 非同步



小明起床後要幾分鐘才能開始吃早餐?

- (A) 25 分鐘
- (B) 19 分鐘
- (C) 15 分鐘
- (D) 10分鐘**

下列程式執行的結果為何

```
setTimeout(function () {  
    console.log('蛋');  
}, 1000);  
setTimeout(function () {  
    console.log('咖啡');  
}, 1000);  
setTimeout(function () {  
    console.log('土司');  
}, 1000);
```

執行結果為何？

1. 等1秒印出蛋，再等1秒印出咖啡，再等1秒印出土司
2. 等1秒，依序同時印出蛋、咖啡、土司
3. 等1秒，不確定蛋、咖啡、土司哪一個先印出
4. 每隔1秒會印出一個項目，但不確定先後順序

下列程式執行的結果為何

```
setTimeout(function () {  
    console.log('蛋');  
}, 3000);  
setTimeout(function () {  
    console.log('咖啡');  
}, 2000);  
setTimeout(function () {  
    console.log('土司');  
}, 1000);
```

執行結果為何？

1. 等1秒印出蛋，再等1秒印出咖啡，再等1秒印出土司(全部印出要3秒鐘)
2. 等1秒印出土司，再等2秒印出咖啡，再等3秒印出蛋(全部印出要6秒鐘)
3. 等1秒印出土司，再等1秒印出咖啡，再等1秒印出蛋(全部印出要3秒鐘)
4. 等3秒，依序印出土司、咖啡、蛋

下列程式執行的結果為何

程式碼如下：

```
setTimeout(function () {  
    console.log('蛋');  
    setTimeout(function () {  
        console.log('咖啡');  
        setTimeout(function () {  
            console.log('吐司');  
        }, 1000);  
    }, 1000);  
}, 1000);
```

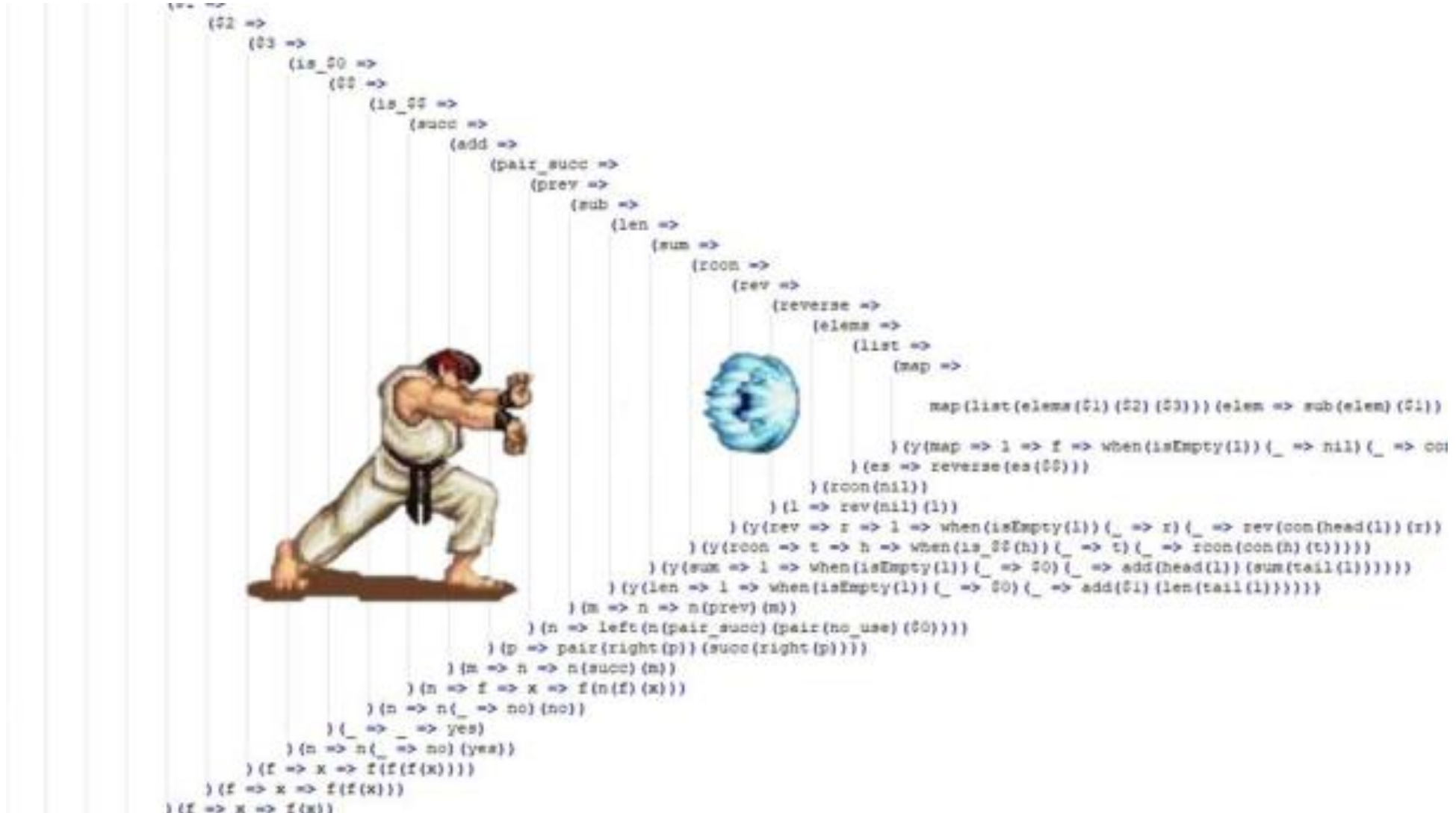


callback hell
pyramid of doom

執行結果為何？

1. 等1秒印出蛋，再等1秒印出咖啡，再等1秒印出吐司(全部印出要3秒鐘)
2. 等1秒印出吐司，再等1秒印出咖啡，再等1秒印出蛋(全部印出要3秒鐘)
3. 等3秒，依序印出蛋、咖啡、吐司
4. 等3秒，依序印出吐司、咖啡、蛋

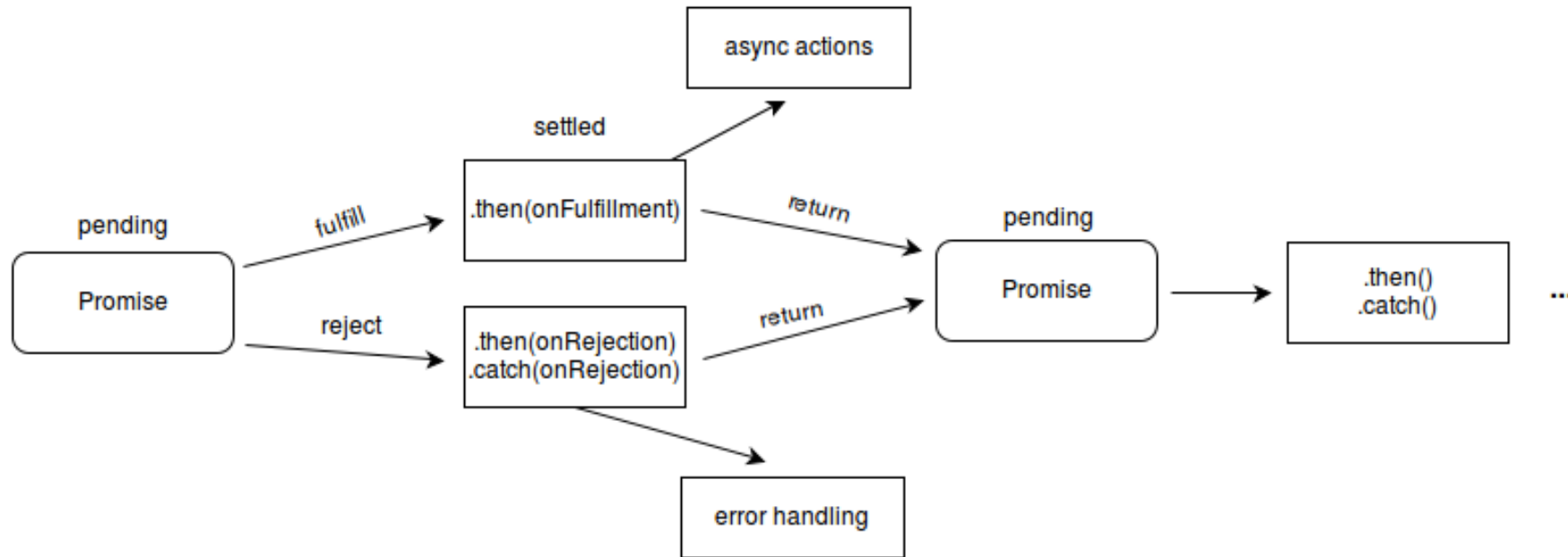
非同步問題 – Callback Hell



圖片來自：<https://www.linkedin.com/pulse/callback-hell-javascript-abir-tasrif-anto>

非同步問題解決之道 - Promise

- promise是一個非同步程式的語法結構，可以解決 Callback hell 的問題
- 表示一個非同步程式運算完的結果



圖片來源：https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

API 參考：<https://w3ctag.github.io/promises-guide/>

promise物件的建立

```
new Promise( /* executor */ function(resolve, reject) { ... } );
```

```
var promise = new Promise(function(resolve, reject) {  
  // 這裡就是原來的非同步(async)、回呼(callback)...的程式  
  if (/* 如果程式執行的結果沒有問題 */) {  
    resolve("沒問題!");  
  }  
  else {  
    reject(Error("錯誤發生了!!"));  
  }  
});
```

Promise 物件的使用

```
promise.then(function(result) {  
    console.log(result); //沒問題!  
}, function(err) {  
    console.log(err); // Error:錯誤發生了!!  
});
```

```
promise.then(function (result) {  
    console.log(result)  
}).catch(function (error) {  
    console.log("error:" + error);  
}).finally(function () {  
    console.log('結束')  
})
```

改用 Promise

- 建立一個函式(function)，回傳 Promise 物件

```
function printSomething(data) {  
  const promise = new Promise(function (resolve, reject) {  
    setTimeout(function () {  
      resolve(data)  
    }, 1000);  
  })  
  return promise;  
}
```

Promise 的鏈式呼叫

- 呼叫回傳 Promise 物件的函式(function)

```
printSomething('蛋')  
  .then((data) => {  
    console.log(data);  
    return printSomething('咖啡');  
  })  
  .then((data) => {  
    console.log(data);  
    return printSomething('土司');  
  }).then((data) => {  
    console.log(data);  
  })
```

Promise語法簡化語法糖 - `async await`

- `async` 定義一個非同步的函式，這個函式回傳一個 `promise`
- `await` 在`async`所定義的`function`中使用，用來等待回傳的`promise`

```
async function asyncFunc() {  
  let data = await printSomething("蛋");  
  console.log(data);  
}
```

你喜歡哪一段程式碼？

(A)

```
蒸蛋(() => {  
  煮咖啡(() => {  
    烤土司(() => {  
      吃早餐()  
    })  
  })  
})
```

(B)

```
蒸蛋()  
  .then(() => 煮咖啡())  
  .then(() => 烤土司())  
  .then(() => 吃早餐())  
  .catch(handleErr)
```

(C)

```
await 蒸蛋()  
await 煮咖啡()  
await 烤土司()  
吃早餐()
```

縮排越來越深，錯誤處理要分層寫

縮排攤平、錯誤處理集中一次寫

長得像同步程式，本質仍是非同步

資料格式

HTML => 資料 + 顯示結構

CSV、XML、JSON、YAML => 資料

CSV

- CSV : Comma Separated Values 以逗號分隔資料

```
id,name,email  
1,Jack,Jack@company.com  
2,Mary,Mary@company.com
```

- TSV : Tab Separated Values 以固定空白分隔資料

```
id name email  
1      Jack      Jack@company.com  
2      Mary      Mary@company.com
```

XML

- XML：Extensible Markup Language(可延伸式的標記語言)
- 用來說明資料是什麼，讓所有應用程式看得懂可以使用

```
<person>
  <firstname>John</firstname>
  <lastname>Smith</lastname>
  <age>25</age>
  <address>
    <streetaddress>21 2nd Street</streetaddress>
    <city>New York</city>
    <state>NY</state>
    <postalcode>10021</postalcode>
  </address>
</person>
```

YAML

- YAML：YAML Ain't Markup Language(不是標記語言)
- 是一種所有語言皆可使用，而且適合人閱讀的的資料格式

```
person:  
  firstname: John  
  lastname: Smith  
  age: 25  
  address:  
    streetaddress: 21 2nd Street  
    city: New York  
    state: NY  
    postcode: 10021
```

JSON

- 目前最主要的資料交換格式

```
{  
  "person":{  
    "firstname":"John",  
    "lastname":"Smith",  
    "age":25,  
    "address":{  
      "streetaddress":"21 2nd Street",  
      "city":"New York",  
      "state":"NY",  
      "postalcode":10021  
    }  
  }  
}
```

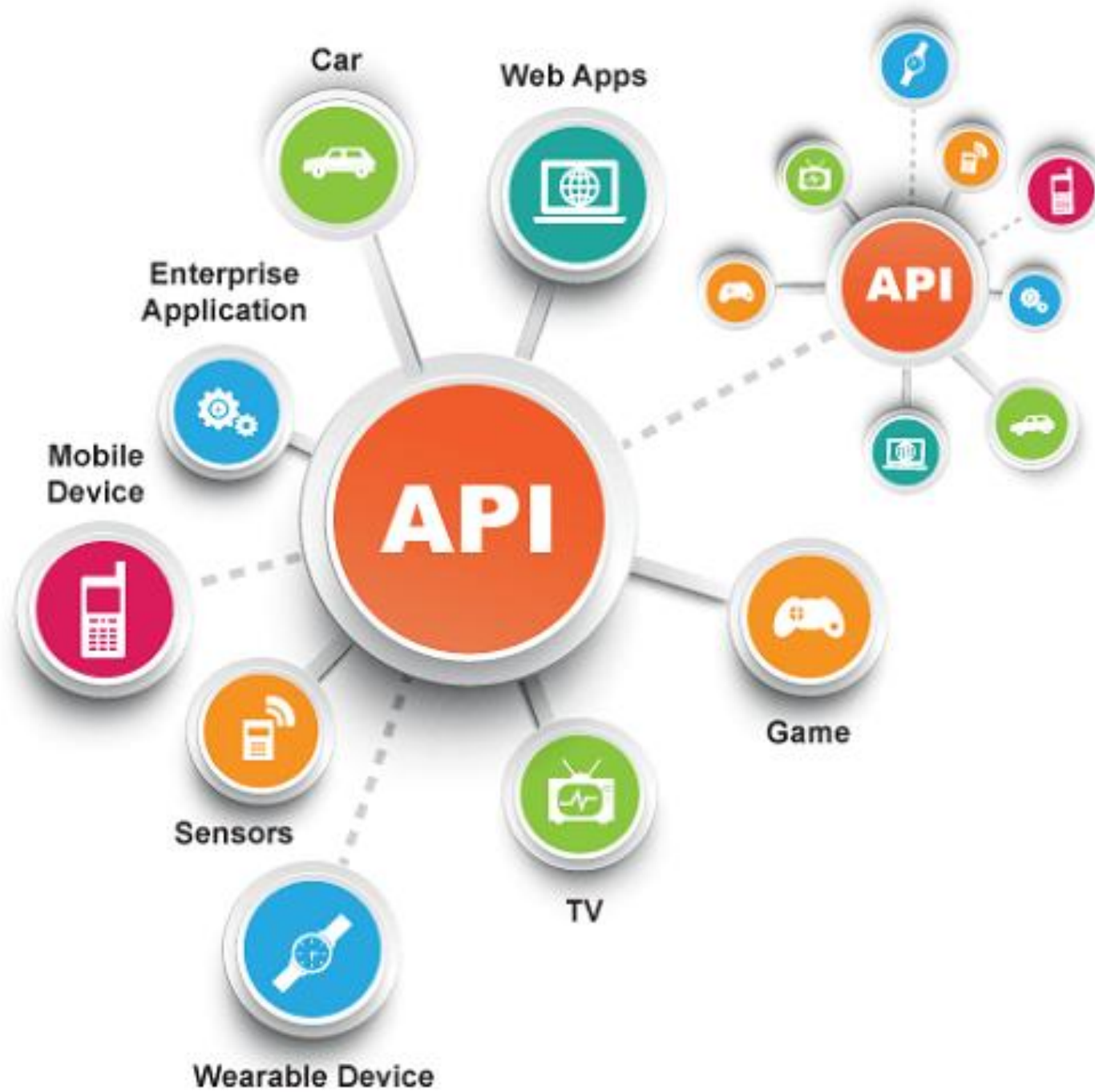
讀取JSON資料

```
const emps = [{"name":"Tom","workYears":3,"salary":35000},  
              {"name":"Jack","workYears":5,"salary":40000},  
              {"name":"Mary","workYears":7,"salary":45000}];
```

```
const emp = {"employees": [  
              {"name":"Tom","workYears":3,"salary":35000},  
              {"name":"Jack","workYears":5,"salary":40000},  
              {"name":"Mary","workYears":7,"salary":45000}  
            ]};
```

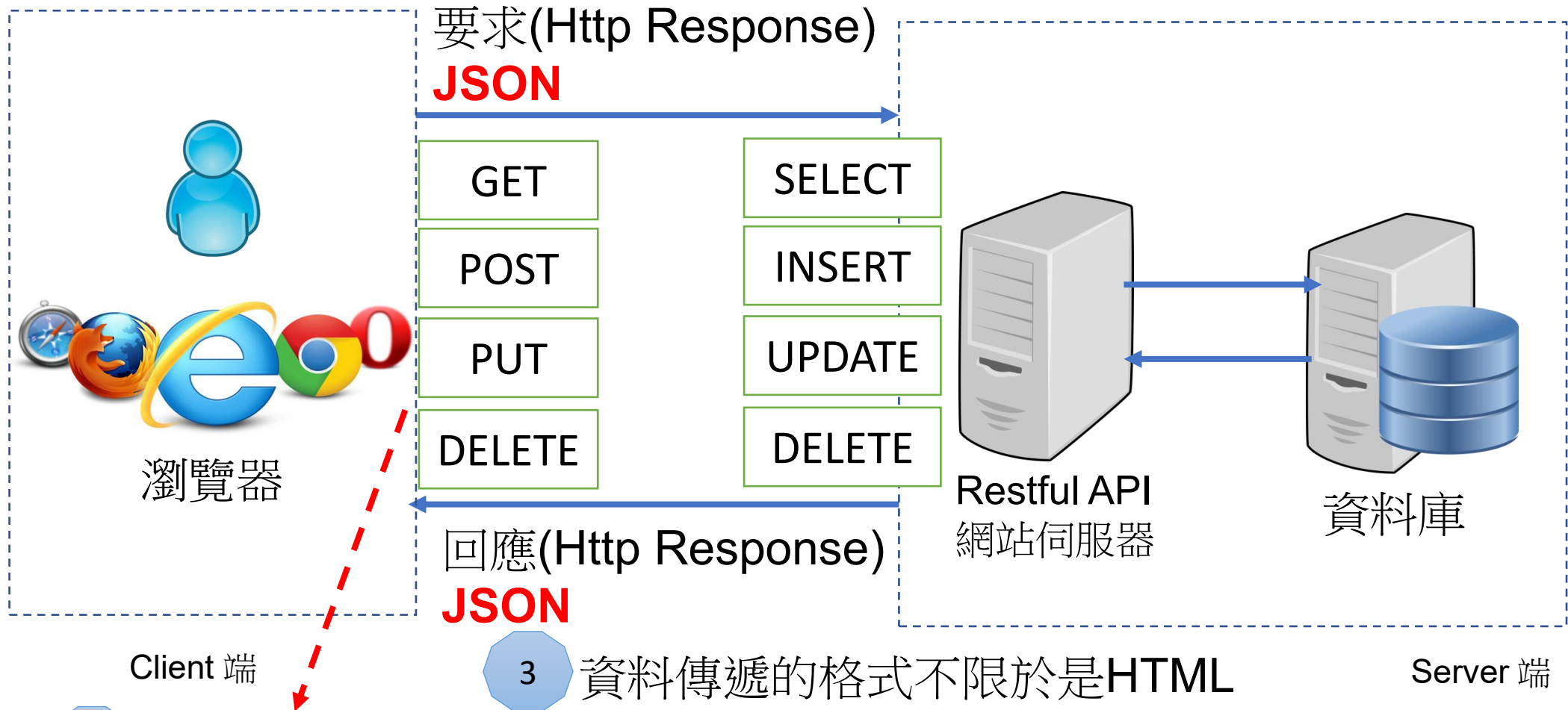
關於 RESTful API

API vs RESTful API



RESTful API

1 獨一無二的網址指定伺服器端的資源：http://localhost/api/shippers



2 透過 HTTP 的定義的方法對資源進行操作

RESTful API URL

- 取得所有資料

HTTP GET `https://localhost/api/shippers`

- 根據id取得單筆資料

HTTP GET `https://localhost/api/shippers/{id}`

- 新增資料，資料或透過

HTTP POST `https://localhost/api/shippers`

- 修改資料

HTTP PUT `https://localhost/api/shippers/{id}`

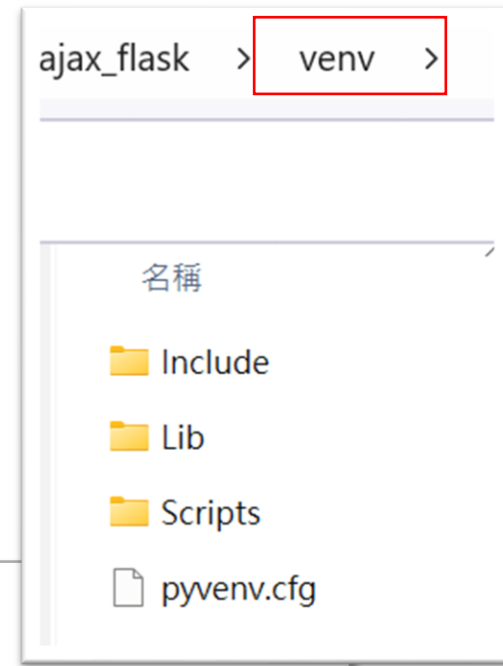
- 刪除資料

HTTP DELETE `https://localhost/api/shippers/{id}`

建立 RESTful API

- 步驟一 建立專案資料夾
- 步驟二 建立虛擬環境

```
C:\workspace\FLASK\ajax_flask> python -m venv venv
```



- 步驟三 進入虛擬環境
 - ✓ 安裝 flask、flask-restful 套件

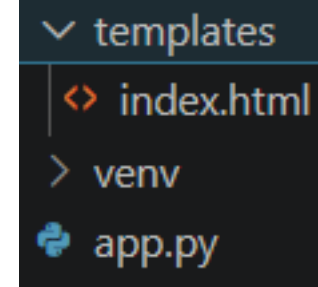
```
C:\workspace\FLASK\ajax_flask> venv\scripts\activate  
(venv) C:\workspace\FLASK\ajax_flask> pip install flask flask-restful
```

步驟四 先建立 FLASK 應用程式

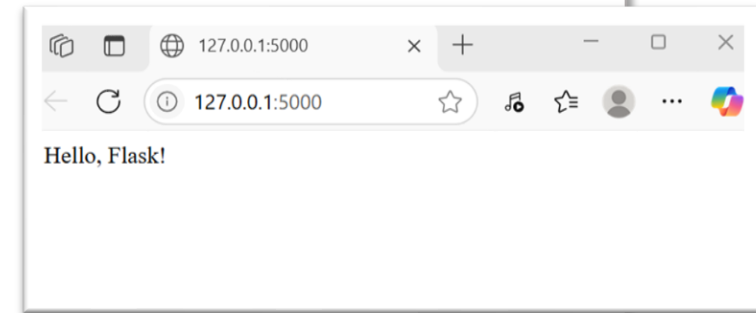
```
from flask import Flask render_template  
app = Flask(__name__)
```

```
@app.route('/')  
def index():  
    return render_template('index.html')
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```



index.html 要放在 templates
資料夾中



```
* Running on http://127.0.0.1:5000  
Press CTRL+C to quit  
* Restarting with stat  
* Debugger is active!  
* Debugger PIN: 114-842-780
```

- 執行 python app.py

步驟五 建立RESTful API

```
from flask import Flask
from flask_restful import Api, Resource
app = Flask(__name__)           # 建立 Flask 物件
api = Api(app)                  # 將 Flask 應用程式轉成 RESTful API 的架構
class HelloWorld(Resource):     # HelloWorld Class 就是一個 API
    def get(self):
        return {'message': 'Hello, RESTful API!'}
api.add_resource(HelloWorld, '/api') # 路由，設定API被呼叫的URL

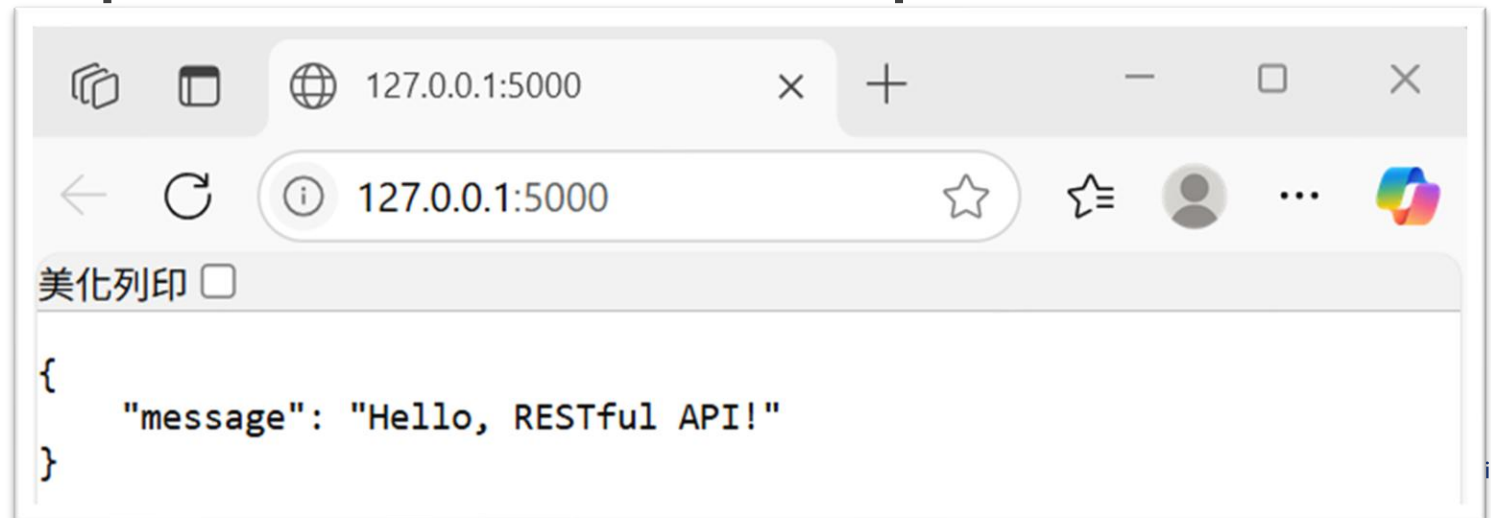
if __name__ == "__main__":
    app.run()
```

執行 API

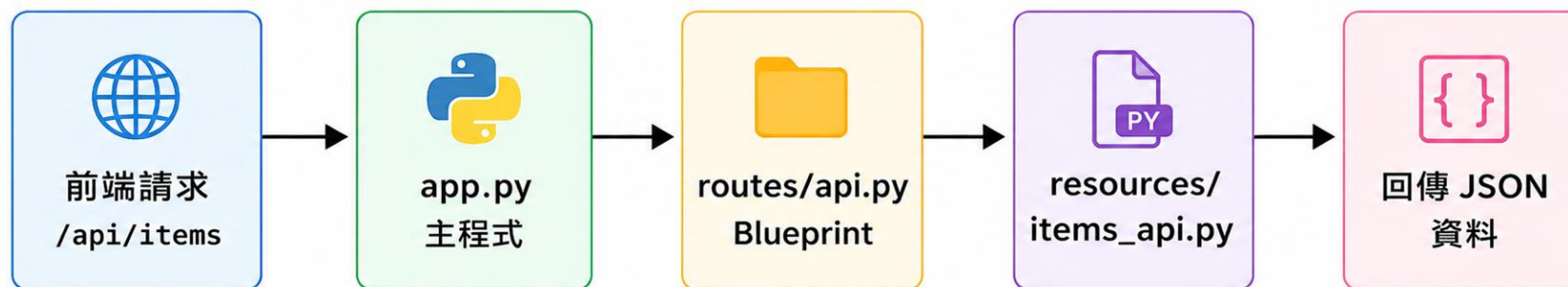
```
(venv) PS C:\workspace\FLASK\api> python app.py
```

```
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PTN: 159-456-434
```

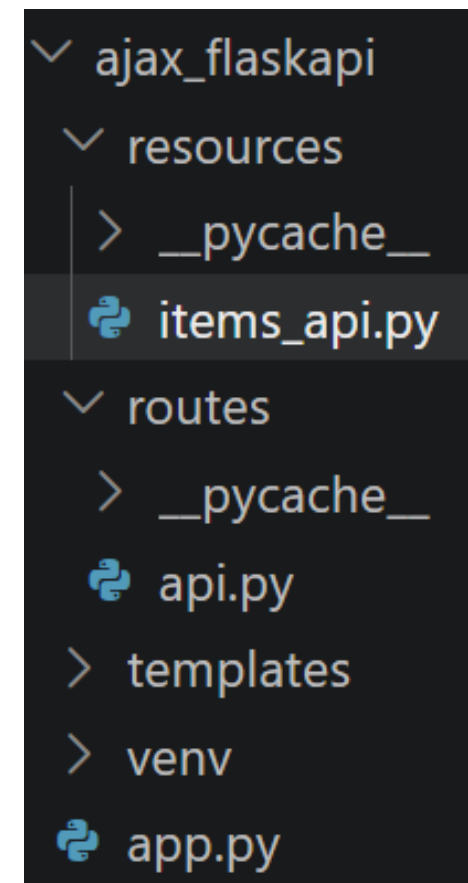
- 開啟瀏覽器輸入 <http://127.0.0.1:5000/api>



上課的 API 架構



- 功能程式碼都放在【resources】資料夾
- Blueprint 是 Flask 的路由管理器，用來管理和分類 API 的請求路徑
- 在主程式中 app.py 引用規劃好的 Blueprint



RESTful API 語法結構

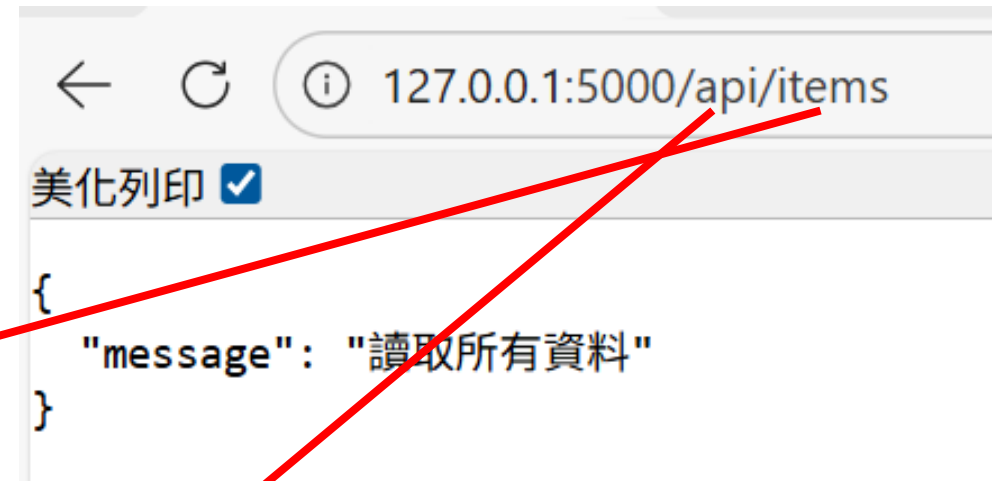
resources/items_api.py

```
from flask_restful import Resource
class Items(Resource):
    def get(self):                                # GET /items
        return {'message': '讀取所有資料'}, 200
    def post(self):                               # POST /items
        return {'message': '新增資料'}, 201
class Item(Resource):
    def get(self, id):                           # GET /items/<id>
        return {'message': f'根據{id}讀取資料'}, 200
    def put(self, id):                           # PUT /items/<id>
        return {'message': f'根據{id}修改資料'}, 200
    def delete(self, id):                        # DELETE /items/<id>
        return {'message': f'根據{id}刪除資料'}, 200
```

flask-restful 路由設定

- 透過 Blueprint 管理 api routes/api.py

```
from flask import Blueprint
from flask_restful import Api
from resources.items_api import Items, Item
api_bp = Blueprint('api', __name__)
api = Api(api_bp)
#設定路由
api.add_resource(Items, '/items')
api.add_resource(Item, '/item/<int:id>')
```

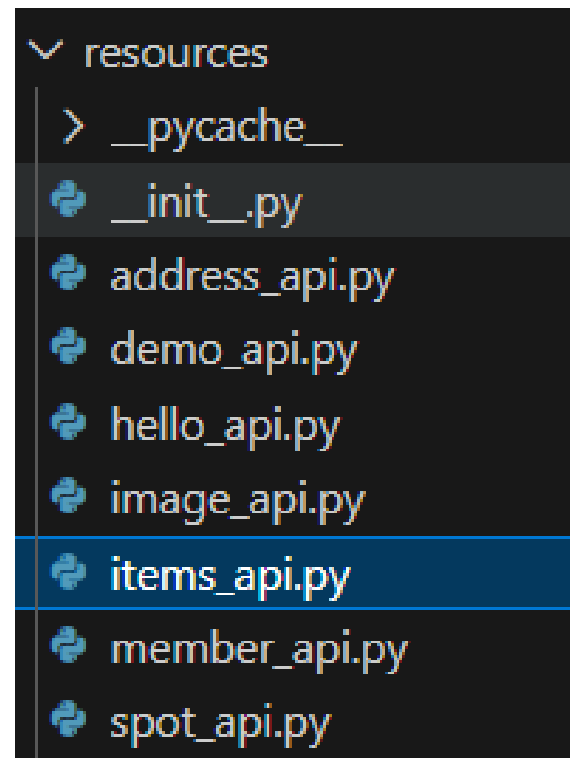


- 在 app.py 中使用 Blueprint

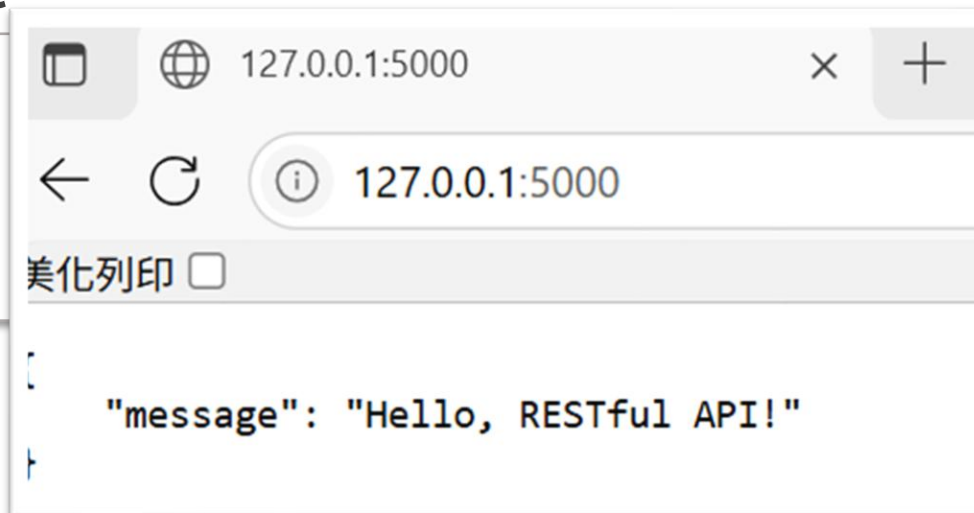
```
from routes.api import api_bp
app.register_blueprint(api_bp, url_prefix='/api')
```

API 架構

- 每個 URL 都會對應到一個資源
- 每一個資源習慣都會獨立成一個 .py 檔
- 統一放在 【resources】 的資料夾中
- 透過 Blueprint 管理 api
- 在主程式中 app.py 引用 Blueprint



```
from routes.api import api_bp
app.register_blueprint(api_bp, url_prefix='/api')
```



測試開發好的 API - Postman

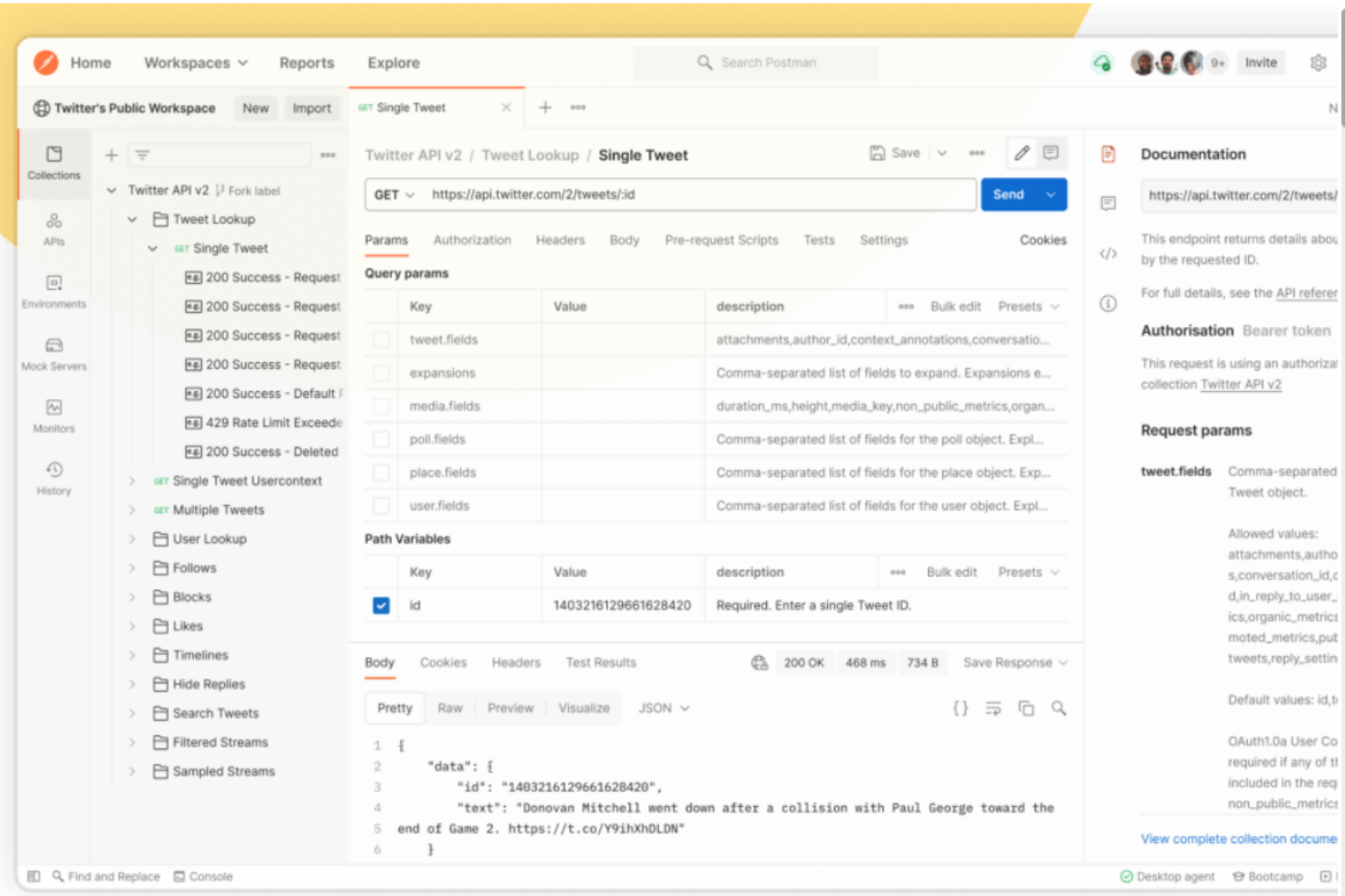


Bu _ APIs together

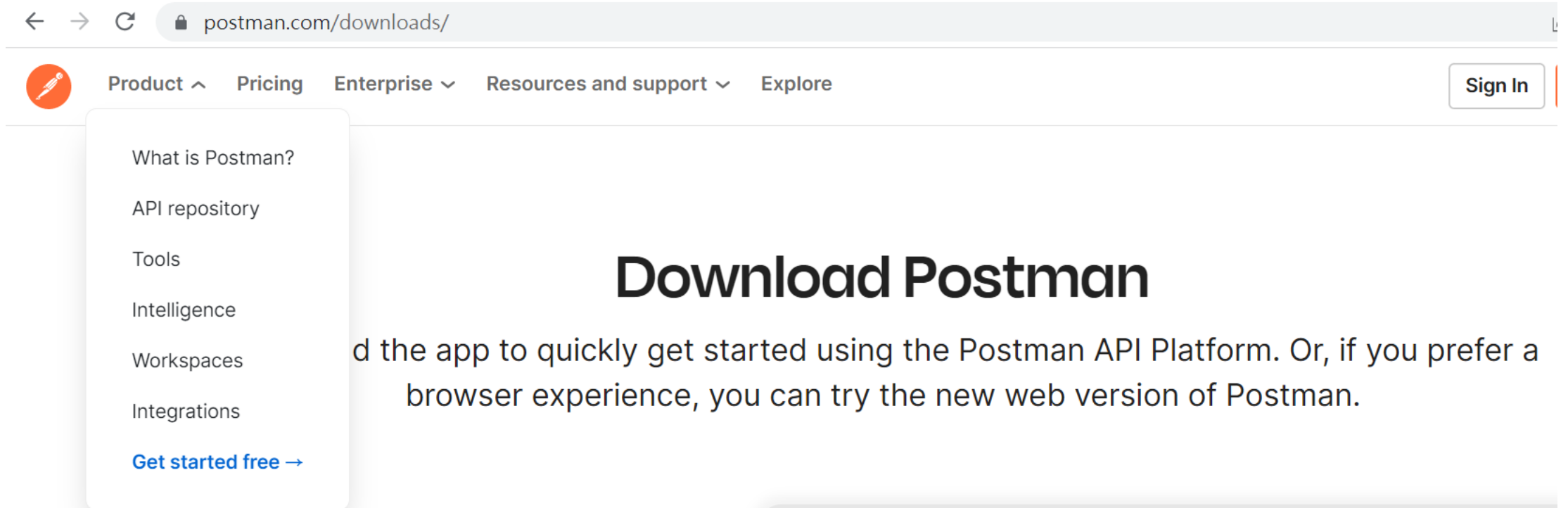
Over 17 million developers use Postman. Get started by signing up or downloading the desktop app.

[Sign Up for Free](#)

Download the desktop app



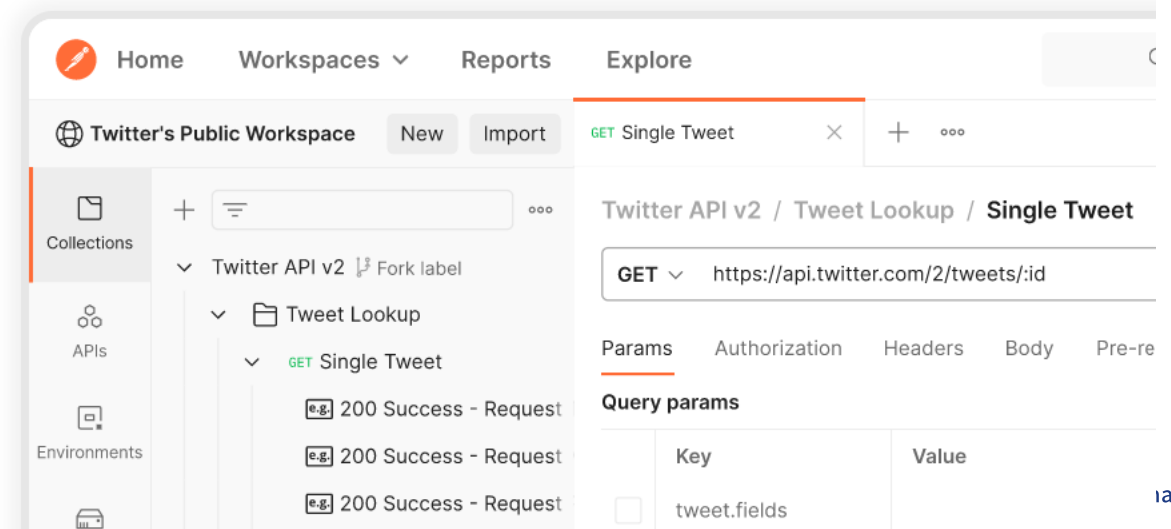
下載 Postman



The Postman app

The ever-improving Postman app (a new release every week) gives you a full-featured Postman experience.

 Windows 64-bit



Postman - 測試 HTTP GET

The screenshot shows the Postman interface for a successful GET request. The method is **GET** and the URL is **http://127.0.0.1:5000/api/items**. The response status is **200 OK** with a response time of **4 ms** and a size of **255 B**. The response body is displayed in the **Pretty** view as a JSON object:

```
{
  "message": "讀取所有資料"
}
```

A red arrow points from the **Send** button to the response body.

Postman - 測試 HTTP POST

The screenshot shows the Postman interface for an HTTP POST request. The request method is **POST** and the URL is `http://127.0.0.1:5000/api/items`. The **Body** tab is selected, and the data type is **JSON**. The request body contains the following JSON:

```
1 {  
2   "name": "tom",  
3   "email": "tom@example.com"  
4 }
```

A red box highlights the request body, and a callout box with the text "要新增的資料" (Data to be added) points to it. A red arrow points from the **Send** button to the response section. The response status is **200 OK** with a response time of **5 ms** and a size of **251 B**. The response body is displayed in the **Body** tab, showing the following JSON:

```
1 {  
2   "message": "新增tom, tom@example.com"  
3 }
```

The response is formatted as **JSON** and is shown in the **Pretty** view.

Postman - 測試 HTTP DELETE

The screenshot shows the Postman interface for testing an HTTP DELETE request. The method is set to **DELETE** and the URL is `http://127.0.0.1:5000/api/items/10`. A red box highlights the method and URL fields. A callout box points to the URL with the text: "這裡要有刪除哪一筆資料的編號". The **Send** button is visible. Below the URL bar, the **Body** tab is selected, showing a JSON response: `{"message": "根據10刪除資料"}`. The status bar indicates a **200 OK** response with a time of **4 ms** and a size of **257 B**. The **Save Response** button is also present.

DELETE `http://127.0.0.1:5000/api/items/10` Send

Params Auth Headers (9) Body ● Pre-req. Tests Settings Cookies

Body `{`
"message": "根據10刪除資料"
`}`

200 OK 4 ms 257 B Save Response

Postman - 測試 HTTP PUT

The screenshot shows the Postman interface for an HTTP PUT request. The method **PUT** is selected and highlighted with a red box. The URL is `http://127.0.0.1:5000/api/items/10`, with the path `/api/items/10` highlighted by a red box and an annotation: "這裡要有修改哪一筆資料的編號" (Here you need to have the ID of the data you want to modify). The **Body** tab is active, showing a JSON body: `{ "name": "tom" }`. An annotation points to the value `"tom"` with the text: "要修改的內容" (Content to be modified). The **Send** button is visible. Below the request, the response is shown with a status of **200 OK**, a time of **4 ms**, and a size of **251 B**. The response body is `{ "message": "根據 10 修改 tom" }`. The response is displayed in the **Pretty** format.

PUT

http://127.0.0.1:5000/api/items/10

Send

Params Auth Headers (9) Body Pre-req. Tests Setting

raw JSON

```
1 {
2   "name": "tom"
3 }
```

要修改的內容

這裡要有修改哪一筆資料的編號

Body

200 OK 4 ms 251 B Save Response

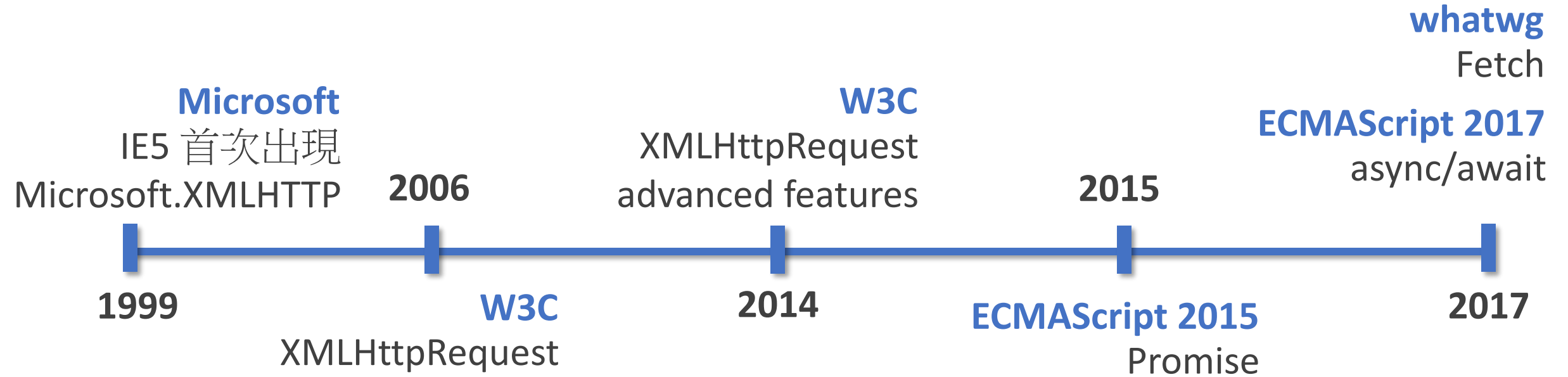
Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "根據 10 修改 tom"
3 }
```

Ajax 程式開發

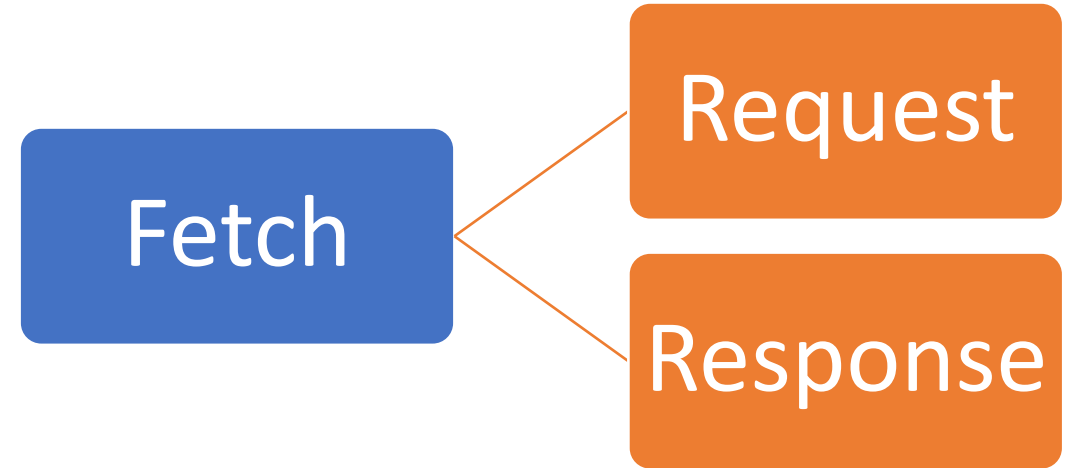
使用 fetch()

技術演進



Fetch API

- XMLHttpRequest物件的替代方案
- 基於 Promise 的語法結構
- 透過全域的fetch()方法，開始發送要求



```
Promise<Response> fetch(RequestInfo input, optional RequestInit init = {});
```

fetch(url)

- 使用 fetch() 得到的一個 Response 物件，包裝在 Promise 結構中

```
const apiUrl = 'http://127.0.0.1:5000/api'
console.log(fetch(apiUrl));
```

```
▼ Promise i
  ► [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
  ► [[PromiseResult]]: Response
```

- 從 Promise 取出 Response 物件，要透過 then() 方法

```
fetch(apiUrl)
  .then(response => {
    console.log(response);
  })
```

取出的 Response 物件

```
Response {type: 'basic', url: 'http:
0, ok: true, ...} i
  body: (...)
  bodyUsed: true
  ► headers: Headers {}
  ok: true
  redirected: true
  status: 200
  statusText: "OK"
  type: "basic"
  url: "http://127.0.0.1:8000/api/"
```

Response 物件相關屬性

```
fetch(apiUrl)
  .then(response => {
    console.log(response);
    console.log(response.headers.get('Content-Type'));
    console.log(response.headers.get('Content-Encoding'));
    console.log(response.headers.get('Date'));
    console.log(response.status);
    console.log(response.url);
    console.log(response.ok);
    console.log(response.body);    //伺服器端回應的內容在這裡
    console.log(response.json()) //promise
  })
```

從 promise 到 async/await

```
const divInfo = document.querySelector('#divInfo');
const callapi = () => {
  fetch('/api/hello')
    .then(response => response.json())
    .then(data => divInfo.textContent = data.message)
    .catch(error => console.error('Error:', error));
}
```

```
const callapi = async () => {
  const response = await fetch(apiUrl)
  const data = await response.json();
  divInfo.innerHTML = `

## ${data.message}</h2>` })


```

api 呼叫失敗

- 先透過 Response 物件中的 ok 屬性，辨別 api 呼叫成功(true)或失敗(false)
- 檢查回傳的資料格式
- 透過 status 回傳的代號，辨別發生了甚麼事
 - 404：路徑錯誤
 - 400：語法錯誤
 - 401：權限不足
 - 403：禁止存取
 - 500：伺服器端程式有錯
 -

```
Response {type: 'basic', url: 'http:  
0, ok: true, ...} {  
  body: (...)  
  bodyUsed: true  
  headers: Headers {}  
  ok: true  
  redirected: true  
  status: 200  
  statusText: "OK"  
  type: "basic"  
  url: "http://127.0.0.1:8000/api/"
```

api 呼叫失敗程式處理範例

//確定 api 回應沒有問題

```
if (response.ok) {  
  const contentType = response.headers.get('content-type');  
  //確定 api 回應的資料為 JSON 格式  
  if (contentType && contentType.includes('application/json')) {  
    const data = await response.json();  
    console.log("成功取得資料:", data);  
    return data;  
  } else {  
    console.error("不是 JSON 格式");  
    throw new Error("Invalid Format");  
  }  
}
```

根據不同的 status 顯示不同的錯誤訊息

```
if (response.status >= 500) {  
    alert('伺服器目前忙碌中，請稍後再試。');  
} else if (response.status === 401 || response.status === 403) {  
    alert('登入逾時或權限不足，請重新登入。');  
} else if (response.status === 404)  
    alert('連線路徑錯誤，請聯繫技術支援。');  
} else {  
    alert('請求發生問題，請確認您的操作是否正確。');  
}
```

最後使用 try...catch 捕捉所有意外

```
try {  
  const response = await fetch(apiUrl);  
  if (!response.ok) throw new Error('有錯誤，無法取得資料');  
  const data = await response.json();  
  divInfo.innerHTML = `

## ${data.message}</h2>` } catch (error) { divInfo.innerHTML = `${error.message}</h2>` } finally { console.log('請求結束') }


```

api 執行太久時的處理

- 下載執行中圖示 <https://icons8.com/preloaders/>

```

```

- 在 try {...} 區塊中顯示執行圖示、停用按鈕功能

```
imgLoader.style.display = 'inline';    //inline、block 顯示  
btn.disabled = true;                  //停用按鈕
```

- 在 finally {...} 區塊中隱藏執行圖示、恢復按鈕功能

```
imgLoader.style.display = 'none';      //隱藏  
btn.disabled = false;                  //啟用按鈕
```

停止 fetch() 呼叫 - AbortController

- AbortController 可以用來取消非同步程式的執行
 - 透過 signal 屬性傳遞取消信號給非同步程式

```
abortController = new AbortController();  
const signal = abortController.signal;  
const response = await fetch(url, { signal })
```

- 透過 abort() 方法，取消非同步程式

```
if (abortController) {  
  abortController.abort()  
}
```

想想看

- 如何透過 `fetch()` 結合 `async/await` 語法進行 api 請求
- 確保程式碼不當掉
 - 確定 api 回傳成功
 - 確定回傳的是 JSON 格式
 - 顯示不同的錯誤訊息
 - 其它意外，透過 `try...catch...finally` 語法處理
- 提升使用者體驗
 - 執行中圖示的顯示與隱藏
 - 按鈕的使用與禁用
- 提供使用者終止請求的能力

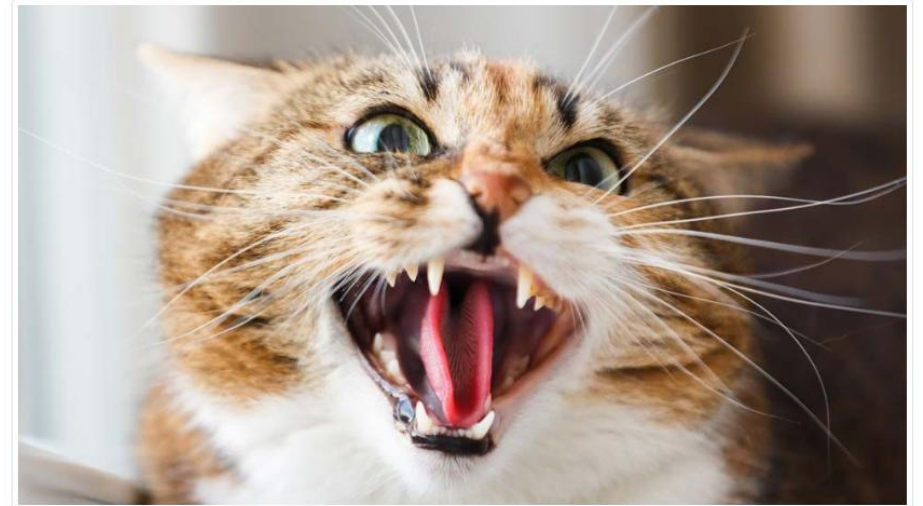
Response 物件 – body

- 伺服器端回應的**資料**包在 body 介面中，需要根據不同的方法，讀取不同的資料，透過這些方法回傳的都是 Promise 物件
 - `arrayBuffer()`、`blob()`、`formData()`、`json()`、`text()`
- 取得伺服器回傳純文字的資料 – `text()`

```
fetch(apiUrl)  
  .then(response => response.text())  
  .then(data => console.log(data))
```

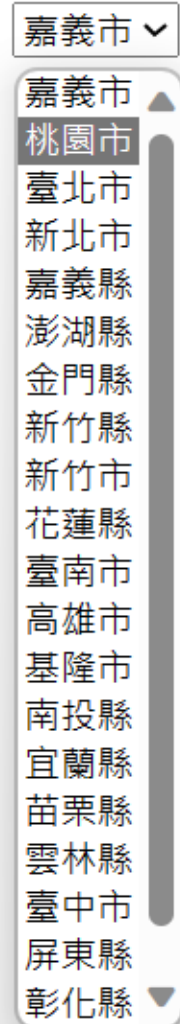
取得伺服器回傳圖片資料 – blob()

```
const myImg = document.querySelector('#img1');  
const apiUrl = 'http://127.0.0.1:5000/api/image';  
(async () => {  
  const response = await fetch(apiUrl)  
  if(!response.ok){  
    console.log(`有錯誤：${response.status}`)  
    return  
  }  
  const datas = await response.blob()  
  myImg.src = URL.createObjectURL(datas)  
})();
```



取得伺服器回傳JSON資料 – json()

```
const apiUrl = 'http://127.0.0.1:5000/api';
(async () => {
  const response = await fetch(`${apiUrl}/cities`);
  const datas = await response.json()
  const cities = datas.map(city => {
    return (`<option value="${city}">${city}</option>`)
  })
  selCity.innerHTML = cities.join("");
})();
```



想想看

- 透過 `fetch()` 呼叫了 `api`，`api` 回傳了一個字串，一張圖片或者是JSON格式的資料，`fetch()` 應該要如何接收？

資料傳遞從瀏覽器到伺服器

Client 資料傳送

- QueryString

```
/api/demo?name=Tom&age=20
```

- Path

```
/api/demo/Tom/20
```

- FormData

```
const formData = new FormData();  
formData.append('name', 'Tom');  
formData.append('age', '20');
```

- JSON

```
const payload = { "name": "Tom", "age": 20};
```

Server 資料接收

```
name = request.args.get('name')
```

```
def get(self, name, age)
```

```
name = request.form.get('name')
```

```
data = request.get_json()  
name = data.get('name')
```

Client 傳遞資料給 API – HTTP GET

- QueryString 查詢字串
 - 將資料放在網址的後面用『?』隔開
 - 資料格式為 key=value，多筆資料時用『&』隔開

GET http://127.0.0.1:8000/api/demo/?name=Tom&age=30

```
const url = `/api/demo/?name=${inpName.value}&age=${inpAge.value}`  
const response = await fetch(url)  
const data = await response.json();  
myDiv.textContent = JSON.stringify(data)
```

讀取表單資料

- FormData 表單資料
 - 是一個 JavaScript 物件
 - 用來收集和發送表單資料
- 將整個表單的資料塞進FormData物件中

```
const formData = new FormData(HTMLFormElement);
```

- 根據需求將資料放進 FormData

```
const formData = new FormData();  
formData.append("userName", "Jack");  
formData.append("password", "1234");
```

FormData 範例

```
<form id="userForm">  
  <div class="mb-3">  
    <label for="userName" class="form-label">姓名 : </label>  
    <input type="text" class="form-control" id="userName" name="Name">  
  </div>  
  ....  
</form>
```

```
const formData = new FormData(document.querySelector('#userForm'));  
for (const pair of formData.entries()) {  
  console.log(` ${pair[0]} `);  
  console.log(` ${pair[1]} `);  
}
```

Name
abc

fetch(url, {⋯})

Promise<Response> fetch(RequestInfo input, optional **RequestInit init = {}**);

● RequestInit (*表示預設值)

```
const response = await fetch(url,{
  method='POST' ,                /*GET、POST、PUT、DELETE
  body:JSON.stringify({}),        // formData
  mode='cors',                   //same-origin、*cors、no-cors
  credentials:'same-origin',      //omit、*same-origin、include
  headers: {'Content-Type': 'application/json'}
  ....
};
```

傳送 FormData 給API – HTTP POST

```
const formData = new FormData(document.querySelector('#userForm'));  
const response = await fetch(apiUrl, {  
    body:formData,  
    method:'POST',  
})  
const data = await response.json()
```

傳送 JSON 資料給 API

```
const response = await fetch(apiUrl,{
  'method':'POST',
  'body': JSON.stringify(searchData),
  'headers': {
    'Content-Type':'application/json'
  }
})
const spots = await response.json()
```

```
const searchData = {
  "categoryId": 0,
  "keyword": "北投",
  "page": 1,
  "pageSize": 9,
  "sortType": "asc",
  "sortBy": ""
}
```

✕ 標頭 承載 預覽 回應 啟動器 時間

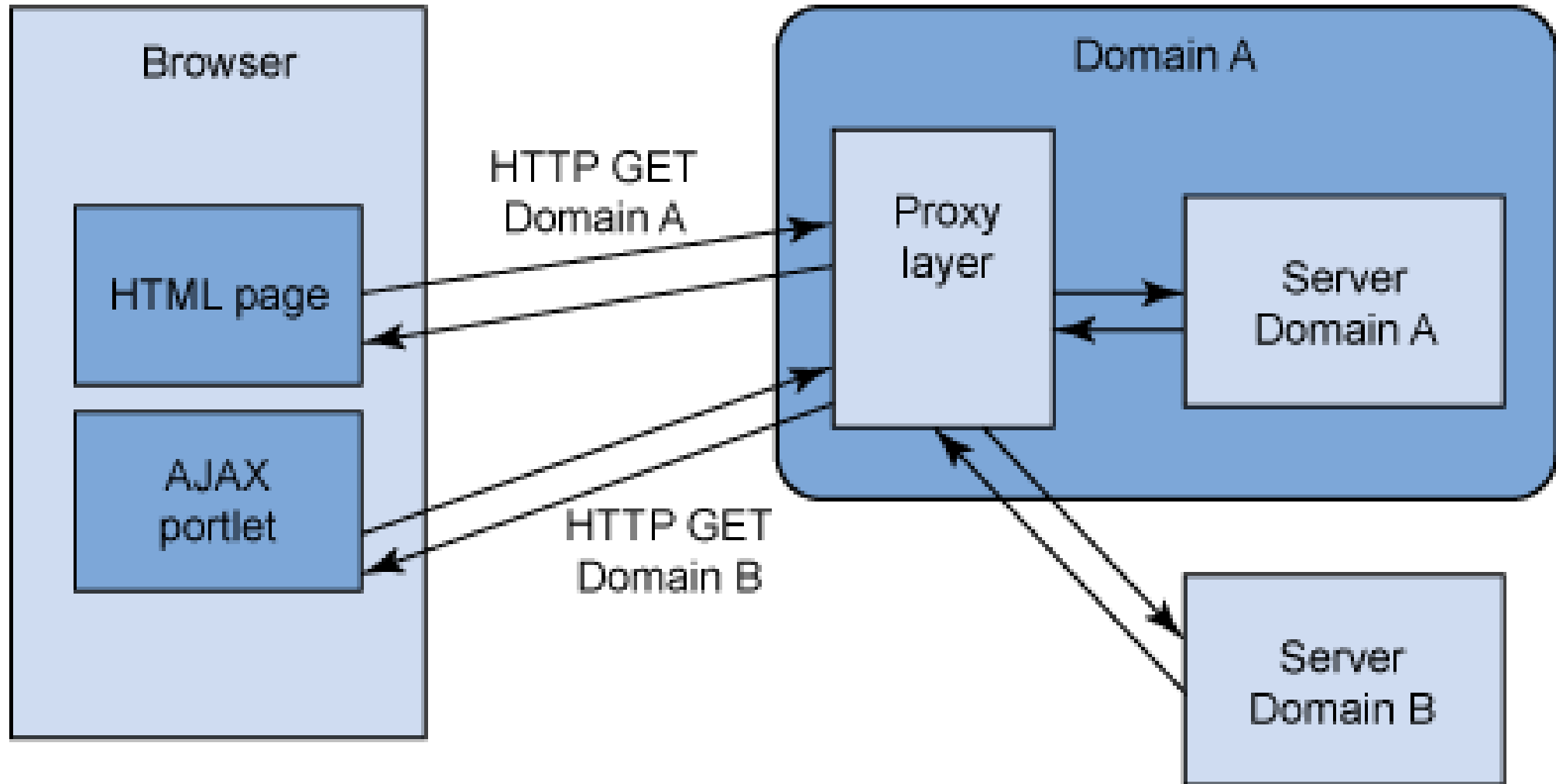
▼ 要求裝載 檢視來源

▼ {categoryId: 0, keyword: "北投", page: 1, pageSize: 9, sortType: "asc", sortBy: ""}

Blocked by CORS policy

✖ Access to fetch at '<http://127.0.0.1:8000/api/category/>' from origin '[index.html:1](#)' 
<http://127.0.0.1:5500>' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. If an opaque response serves your needs, set the request's mode to 'no-cors' to fetch the resource with CORS disabled.

如何取得別的網域的資料



同源政策 Same Origin Policy

- 基於安全性的理由,限制瀏覽器端的程式,例如 JavaScript 不能請求來源不同的資料,但是對於 HTML 標籤(Tag) 沒有此限制
- 這裡所講的[來源], 指的是網域(網址)
http://www.iiiedu.org.tw:80/course
protocol://host.domain:port/path
- 只要 protocol, host.domain, port 有一項不同,即為不同

相同或不同網域範例

Origin : **http://www.example.com/dir/page.html**

Compared URL	Outcome	Reason
http://www.example.com/dir/page.html	Success	Same protocol and host
http://www.example.com/dir2/other.html	Success	Same protocol and host
http://www.example.com: 81 /dir/other.html	Failure	Same protocol and host but different port
https ://www.example.com/dir/other.html	Failure	Different protocol
http:// en.example.com /dir/other.html	Failure	Different host
http:// example.com /dir/other.html	Failure	Different host (exact match required)
http:// v2.www.example.com /dir/other.html	Failure	Different host (exact match required)

怎麼做?讓JavaScript跨網域

- 跨來源資源共享(CORS) Cross-Origin Resource Sharing
 - 允許Client端程式,呼叫其它網域的程式
 - <https://fetch.spec.whatwg.org/>
- Server端允取跨網域呼叫的程式要加上這段程式

//指定要求資源時，允許可以使用的來源(網址)

Access-Control-Allow-Origin: <origin> | *

//指定要求資源時，允許使用的http方法：GET、POST、PUT、Delete

Access-Control-Allow-Methods: <method>[, <method>]*

//指定要求資源時，允許可以使用的 header

Access-Control-Allow-Headers: <field-name>[, <field-name>]*

Flask 允許跨來源呼叫

- 安裝套件

```
pip install flask-cors
```

- 允許跨來源呼叫設定

```
from flask_cors import CORS  
app = Flask(__name__)  
# 對 app 套用 CORS  
CORS(app)
```

練習看看1

- 剛剛的練習載入了縣市資料
- 現在要做的是
 - 選擇某個縣市後帶出[鄉、鎮、區]
 - 選擇某個鄉鎮區帶出[路]
- 資料庫address資料表的資料來自
 - <https://data.gov.tw/dataset/35321>

👁 瀏覽次數: 24209 📄 下載次數: 10163 💬 意見數: 24

主要欄位說明 *粗體欄位為資料標準欄位	city(縣市名稱)、site_id(行政區域名稱)、road(路名)
資料資源下載網址	📄 CSV 檢視資料 109全國路名資料 📄 API 檢視資料 109全國路名資料-URL存取JSON格式

南投縣 ▾

中寮鄉 ▾

中集路 ▾

南投縣 ▲

嘉義市

嘉義縣

基隆市

宜蘭縣

屏東縣

彰化縣

新北市

新竹市

新竹縣

桃園市

澎湖縣

臺中市

臺北市

臺南市

臺東縣

花蓮縣

苗栗縣

金門縣

雲林縣 ▾

AjaxSite - Privacy

練習看看2

1 2 3 4 5 6 7 8 9 10

景點搜尋



太平山國家森林遊樂區

蹦蹦車、溫泉、高山湖泊、巨木森林與國寶山毛櫸，鋪成了太平山國家森林遊樂區超過百年歷史的綿長軌跡。太平山舊稱「眠腦」，是泰雅族語「鬱鬱蒼蒼」之意。1914年，日本人先做了資源調查，1915年決議開發

宜蘭縣大同鄉太平巷58之1號



滿月圓國家森林遊樂區

在三峽大豹溪的上游，一條叫做蚬子溪的支流上，座落著一個擁有瀑布、生態與楓紅的美麗小天地：滿月圓國家森林遊樂區。距離三峽市區約40分鐘的滿月圓國家森林遊樂區，海拔在300至1700公尺之間，終年潮濕

新北市三峽區有木里有木174-1號



內洞國家森林遊樂區

瀑聲隆隆，依山傍水的蕨類樂園，小巧精緻的內洞國家森林遊樂區，是南勢溪上游一個享受寧靜與綠意的好所在。距新店約50分鐘，內洞可說是大台北地區交通最方便的國家森林遊樂區，海拔在230~1500公尺之間

新北市烏來區信賢里娃娃谷46號



東眼山國家森林遊樂區

距離三峽市區約40分鐘的東眼山國家森林遊樂區，海拔在300至1700公尺之間，終年潮濕



觀霧國家森林遊樂區

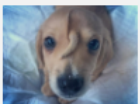
距離三峽市區約40分鐘的觀霧國家森林遊樂區，海拔在300至1700公尺之間，終年潮濕



拉拉山國家森林遊樂區

拉拉山國家森林遊樂區位於桃園市復興區，海拔在230~1500公尺之間

練習看看3

	姓名	Email	年齡	
	Jack	Jack@gmail.com	25	 
	Mary	Mary@gmail.com	31	 
	Nancy	Nancy@gmail.com	34	 
	Tom	Tom@company.com	18	 
	Eric	Eric@company.com	29	 
	Thomas	Thomas@gmail.com	36	 
	Jimmy2	Jim@company.com	30	 

姓名

電子郵件

密碼

密碼確認

年紀

Avatar

選擇檔案

沒有選擇檔案

練習看看4

選擇圖片

選擇檔案

Island.jpg



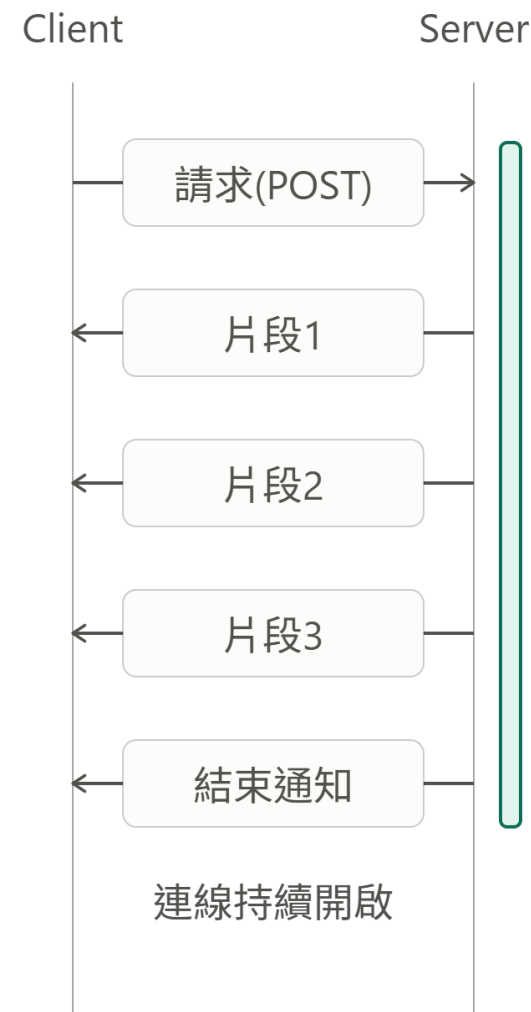
開始辨識

辨識結果

```
{
  "AttractionName": "花瓶岩",
  "Country": "台灣",
  "City": "屏東縣",
  "Town": "琉球鄉",
  "Description": "小琉球花瓶岩是台灣屏東縣琉球鄉的著名地標，因其特殊的壺狀珊瑚礁岩形狀，頂端長滿綠色植物，宛如插著花的花瓶而得名。此處海水清澈，是熱門的浮潛及水上活動地點。",
  "Latitude": "22.355083",
  "Longitude": "120.380721"
}
```

Server Sent Event (SSE)

- 瀏覽器和伺服器之間單向傳送協定，資料只能由伺服器傳送到用戶端
- 應用在即時新聞、股票價格及即時匯率等，瀏覽器只需要單純的顯示資訊的功能上
- SSE 功能的建立
 - SSE Client：EventSource API
 - SSE Server：回傳符合 Event-Stream 格式的資料



SSE Server - 推送資料格式

- Response Content-Type 需為 text/event-stream
- 資料由下列格式所組成，需為UTF-8編碼

```
: 這是註解 \n
event : show \n           //自訂事件名，取代 message
id : 1 \n                   //最後一次觸發事件的ID值
retry : 5000 \n              //斷線後 5 秒重連
data : 資料1 \n \n         //必要的資料
```

\n\n 兩個換行代表「一則事件結束」

SSE Server 程式範例

```
import time
from datetime import datetime
from flask import Response, stream_with_context

def clock_stream():
    @stream_with_context
    def event_stream():
        try:
            while True:
                now = datetime.now().strftime("%H:%M:%S")
                yield f"data: {now}\n\n"
                time.sleep(1)          # 每隔一秒推一次
        except GeneratorExit:
            pass

    return Response(
        event_stream(),
        mimetype="text/event-stream",
    )
```

SSE Client – EventSource API

- EventSource 物件，url 是提供資料的伺服器端程式

```
source = new EventSource( url)
```

- 相關屬性事件
 - onopen、onerror
 - onmessage：用來接收伺服器端程式回傳的結果

SSE Client 程式範例

```
<h1 id="clock" class="display-3 my-2">--:--:--</h1>
```

```
<script>
```

```
  const clock = document.querySelector('#clock');
```

```
  let source = null;
```

```
  function connect() {
```

```
    // 建立與伺服器的 SSE 連線
```

```
    source = new EventSource('/api/clock');
```

```
    // 根據伺服器指定的事件名稱來接收資料，預設是 message
```

```
    source.addEventListener('show', event => {
```

```
      clock.textContent = event.data;
```

```
    });
```

```
  }
```

```
</script>
```

16:24:35

附錄(自行參考)

上課用資料庫- SQLite

- 透過 DB Browser for SQLite 開啟專案中的 mydb.db



- DB Browser for SQLite下載 <http://sqlitebrowser.org/>

SQLAlchemy

- 是 Python 中知名的 ORM 框架
 - ORM 是 Object Relational Mapper 的縮寫
 - 可以不用寫 SQL 語法完成資料庫的CRUD
 - 資料庫系統改變時可以不用動到程式碼
-
- 安裝套件

```
pip install flask-sqlalchemy
```

建立 SQLAlchemy 物件

- models/__init__.py

```
from flask_sqlalchemy import SQLAlchemy
```

```
# 建立 SQLAlchemy 物件
```

```
db = SQLAlchemy()
```

- 彙整所有模型(Model)

```
from .member_model import Member
```

```
from .spot_model import Spot
```

```
from .address_model import Address
```

```
from .category_model import SpotCategory
```

設定資料庫連線

● app.py

```
# 取得 app.py 所在位置的實際路徑
```

```
basedir = os.path.abspath(os.path.dirname(__file__))
```

```
# 定義要連到 my.db sqlite 資料庫的連線字串
```

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:/// ' + os.path.join(basedir, 'my.db')
```

```
# 關掉一些監控功能，提升效能
```

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

```
# 初始化資料庫連線
```

```
db.init_app(app)
```

建立模型

- models/member_model.py

```
from . import db

class Member(db.Model):
    __tablename__ = 'members'

    MemberId = db.Column(db.Integer, primary_key=True)
    Name = db.Column(db.Text)
    Email = db.Column(db.Text)
    Age = db.Column(db.Integer)
    FileName = db.Column(db.Text)
    Password = db.Column(db.Text)
```

Flask RESTful api 的 CRUD

● 基本架構 resources/member_api.py

```
from flask import request
from flask_restful import Resource
from models import db
from models.member_model import Member
from sqlalchemy.exc import SQLAlchemyError
```

```
class MembersResource(Resource):
    def get(self):
        """讀取所有會員"""
        pass

    def post(self):
        """新增會員"""
        pass
```

```
class MemberResource(Resource):
    def get(self, id):
        """讀取單筆會員"""
        pass

    def put(self, id):
        """修改單筆會員"""
        pass

    def delete(self, id):
        """刪除單筆會員"""
        pass
```

讀取所有會員資料

```
def get(self):  
    """讀取所有會員"""  
    # 從資料庫讀取所有會員資料，放進 members 變數中  
    members = Member.query.all()  
  
    # 串列解析式 (List Comprehension) [{ ... } for m in members]  
    # 將物件轉為字典清單後，直接 return 這個清單， Flask-RESTful 會幫你轉成 JSON  
    return [{  
        "MemberId": m.MemberId,  
        "Name": m.Name,  
        "Email": m.Email,  
        "Age": m.Age,  
        "FileName": m.FileName  
    } for m in members], 200
```

根據會員編號讀取單筆資料

```
def get(self, id):  
    """讀取單筆會員"""  
    member = Member.query.get(id)  
    if not member:  
        return {"message": "找不到該成員"}, 404  
    return {  
        "MemberId": member.MemberId,  
        "Name": member.Name,  
        "Email": member.Email,  
        "Age": member.Age,  
        "FileName": member.FileName  
    }, 200
```

新增會員資料

```
def post(self):
    """新增會員 (包含檔案與資料)"""
    name = request.form.get("Name") #接收從瀏覽器傳過來的資料
    email = request.form.get("Email")
    # 將接收到的資料放進 Member 物件中
    new_member = Member(
        Name=name,
        Email=email,
    )

    try:
        db.session.add(new_member) # 先將資料放進暫存區
        db.session.commit() # 將暫存區的資料新增到資料庫
        return {"message": "成功建立會員", "id": new_member.MemberId}, 201
    except SQLAlchemyError as e:
        db.session.rollback() # 資料庫異動如果有錯，將所有的異動還原
        return {"message": "資料庫寫入失敗", "error": str(e)}, 500
```

修改會員資料

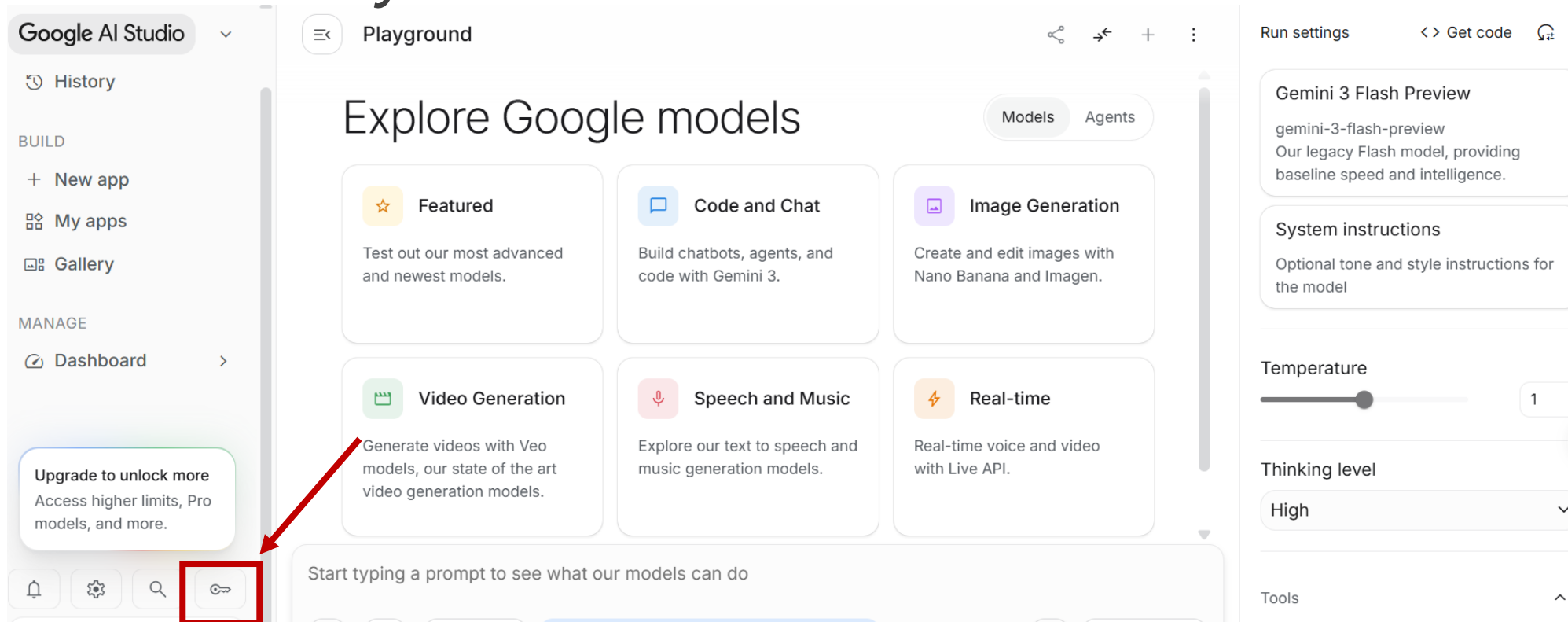
```
def put(self, id):  
    """修改單筆會員"""  
    member = Member.query.get(id) # member 放的是你要更新的資料  
    if not member:  
        return {"message": "找不到該成員"}, 404  
  
    # 接收從瀏覽器傳過來的資料，更新到 member 中  
    member.Name = request.form.get("Name", member.Name)  
    member.Email = request.form.get("Email", member.Email)  
    member.Age = request.form.get("Age", member.Age)  
  
    try:  
        db.session.commit()  
        return {"message": "資料更新成功"}, 200  
    except SQLAlchemyError:  
        db.session.rollback()  
        return {"message": "更新失敗"}, 500
```

刪除會員資料

```
def delete(self, id):  
    """刪除單筆會員"""  
    member = Member.query.get(id)  
    if not member:  
        return {"message": "找不到該成員"}, 404  
  
    try:  
        db.session.delete(member)  
        db.session.commit()  
        return {"message": f"會員 {id} 已刪除"}, 200  
    except SQLAlchemyError:  
        db.session.rollback()  
        return {"message": "刪除失敗"}, 500
```

取得 Gemini API Key(1)

- 要有 google 帳號，登入 Google AI Studio
<https://aistudio.google.com/>
- 建立 API Key



取得 Gemini API Key(2)

● 點選【Create API Key】

The screenshot shows the Google AI Studio interface. On the left is a sidebar with navigation options: Google AI Studio, Dashboard, PROJECT (API Keys, Projects), USAGE & BILLING (Usage, Rate Limit, Spend, Billing), and OBSERVE (Logs and Datasets). The main area is titled 'API Keys' and includes a 'Group by' dropdown (set to 'API Key'), a 'Filter by' dropdown (set to 'All projects'), and a table with columns 'Key', 'Project', 'Created', and 'Billing Tier'. A red box highlights the 'Create API key' button in the top right. A red arrow points from this button to a modal window titled 'Create a new key'. The modal contains a 'Name your key' text input, a 'Choose an imported project' dropdown menu (showing 'Select a Cloud Project'), and 'Cancel' and 'Create key' buttons. A blue box on the right contains the text '建立專案名稱' and '建立 Key 名稱'. At the bottom of the page, there is a link that says 'Can't find your API keys here?'.

Google AI Studio

< Dashboard

PROJECT

API Keys

Projects

USAGE & BILLING

Usage

Rate Limit

Spend

Billing

OBSERVE

Logs and Datasets

API Keys

Group by API Key Project

Filter by All projects

Key Project Created Billing Tier

...qR_w key2026

Create a new key

Name your key

Choose an imported project

Select a Cloud Project

Cancel Create key

Set up billing Free tier

建立專案名稱
建立 Key 名稱

Can't find your API keys here?

在 Python 中使用 API Key

- 建立 .env 檔，在此檔案中放 API Key，不要把此檔案上傳到 GitHub

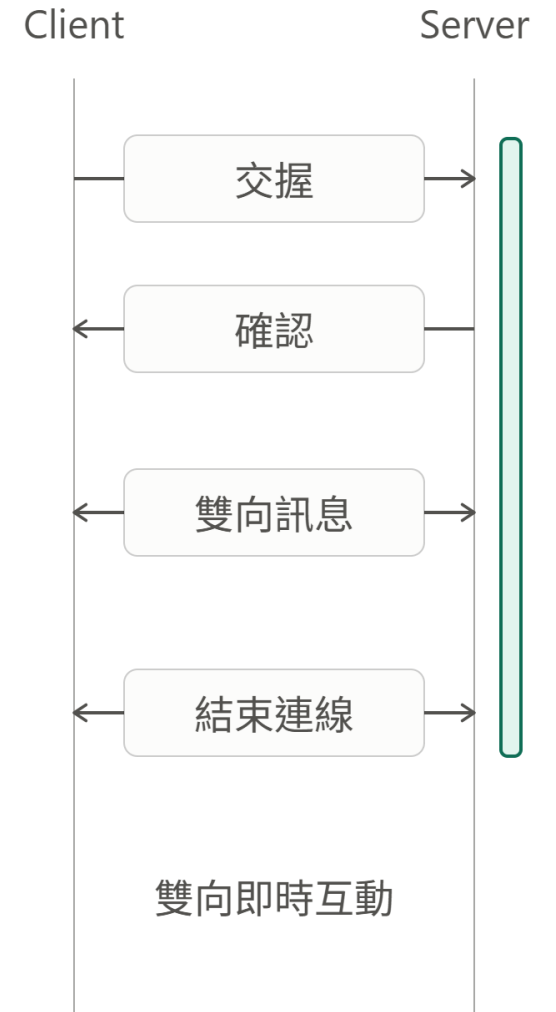
GEMINI_API_KEY= APIKEY字串

- 在 Python 中讀取 .env 檔中的資料
 - 先安裝套件 `pip install python-dotenv`

```
import os
from dotenv import load_dotenv
load_dotenv() # 讀取 .env 檔案
gemini_api_key = os.getenv('GEMINI_API_KEY')
```

WebSocket

- WebSocket 協定是一種在 Web 應用中實現即時和雙向通信的重要標準
- 架構在HTTP/HTTPS協定上，可以使用 wss:// 實現加密通信
- 應用在聊天室、線上遊戲及系統監控等，需要透過長時間連線的功能上
- WebSocket功能的建立
 - WebSocket Client：HTML5 中有提供 WebSocket API
 - WebSocket Server：不同的 Server 有不同的實作（例如 Flask 可搭配 flask-sock）



WebSocket API

- WebSocket 物件，URL協定需是 ws:// 或 wss://

```
socket = new WebSocket(url [, protocols ])
```

- 相關屬性事件
 - onopen、onclose、onerror
 - onmessage：用來接收 WebSocket Server 回傳的結果
- 傳遞資料到WebSocket Server的方法
 - send(data) //string、Blob、ArrayBuffer

WebSocket API(一)

- 步驟一 建立 WebSocket 物件

```
const host = "wss://echo.websocket.org/";  
const socket = new WebSocket(host);
```

- 步驟二 連線如果成功，發送訊息給 WebSocket Server

```
socket.addEventListener('open', event => {  
  socket.send("Hello Socket!!");  
})
```

WebSocket API(二)

- 步驟三 在message事件中，接收 WebSocket Server 回傳的資料

```
socket.addEventListener('message', event => {  
  console.log(`來自於 SocketServer 的資料： ${event.data}`);  
  socket.close(); //關閉連線  
})
```

- 步驟四 連線關閉

```
socket.addEventListener('close', event => {  
  console.log("連線關閉!!");  
})
```

Flask 建立 WebSocket Server(一)

- 安裝套件

```
pip install flask-sock
```

- 在 Blueprint 中把WebSocket 的路由集中註冊在一起

```
from flask_sock import Sock  
from resources.echo_ws_api import init_echo_ws  
sock = Sock(api_bp)  
init_echo_ws(sock)
```

Flask 建立 WebSocket Server(二)

● 最簡單的 Echo Server

```
def init_echo_ws(sock):
```

```
    @sock.route("/ws/echo")
```

```
    def echo(ws):
```

```
        while True:
```

```
            message = ws.receive()
```

```
            if message is None:
```

連線關閉

```
                break
```

```
            ws.send(f"Echo: {message}") # 原訊息加個Echo:再送回瀏覽器
```

→ send: Hello WebSocket!!

← message: Echo: Hello WebSocket!!

- ws.receive() 接收 WebSocket Client 傳過來的資料
- ws.send(data) 傳送資料給 WebSocket Client

Thank you